Machine Learning

PHYS 453

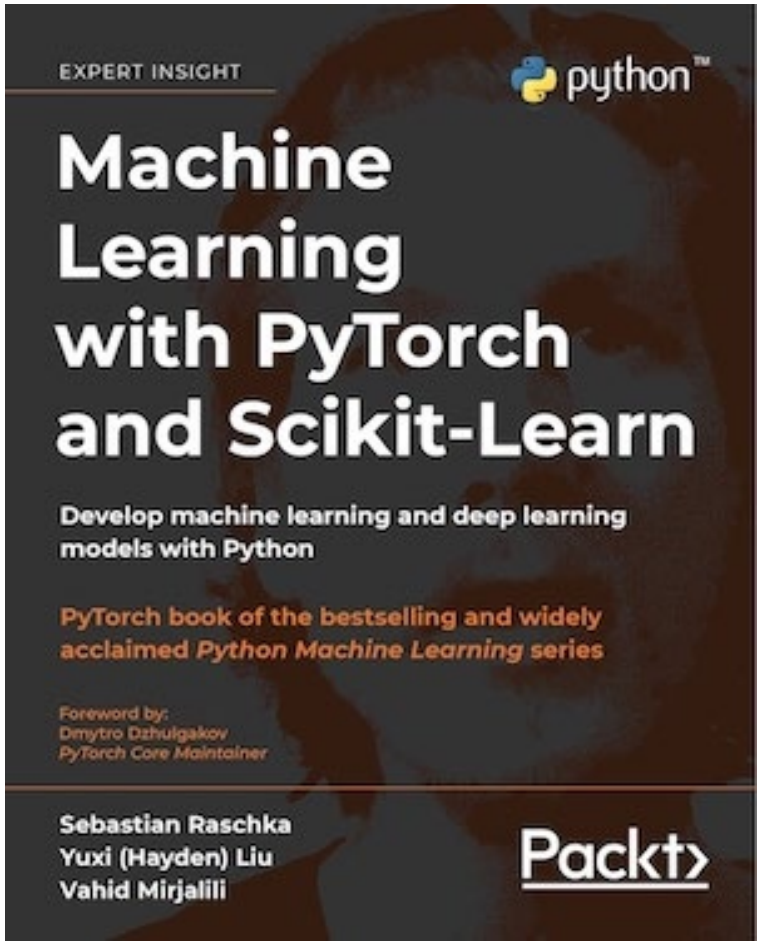
Dr. Daugherity

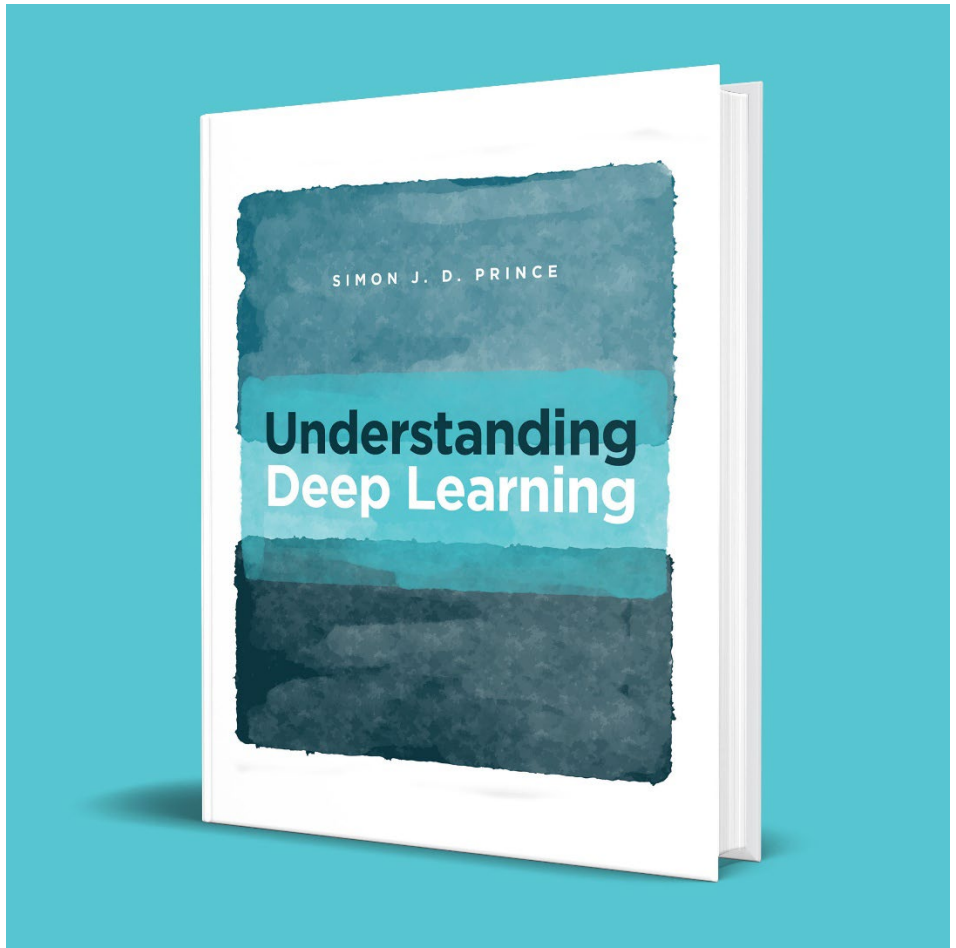
Deep Learning

- Given tools in pytorch, what can we do with all of this power?
- Quick tour through some different layers and topologies

Sources

- https://github.com/rasbt/machine-learning-book/tree/main
- https://udlbook.github.io/udlbook/

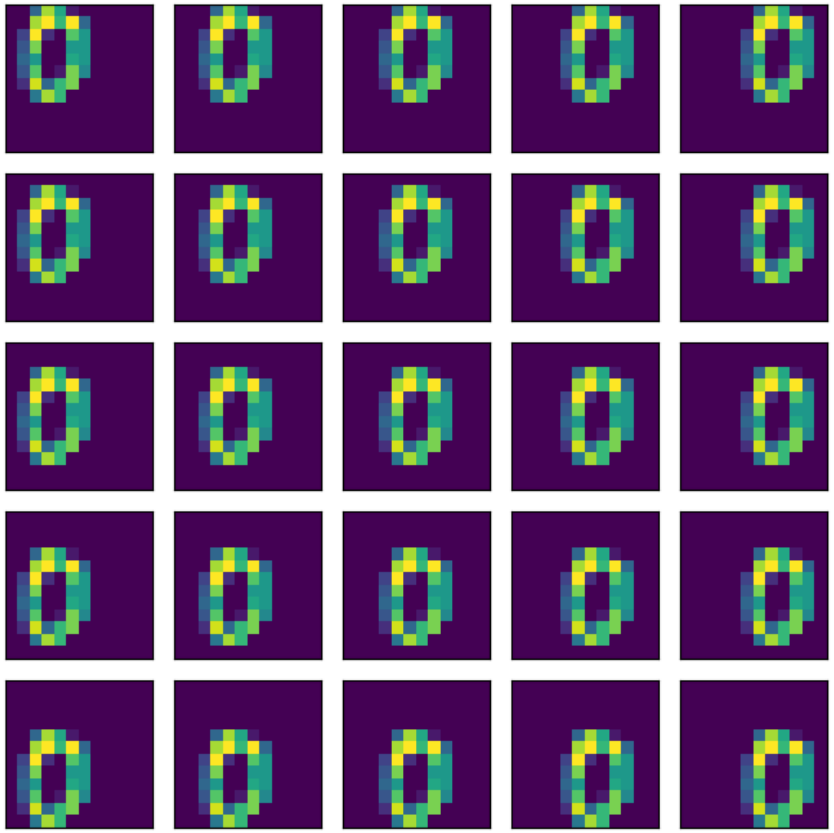




CONVOLUTIONS

Convolutions

- Sometimes we need patterns in nearby features
- Example: a zero is a zero no matter where it occurs in the image



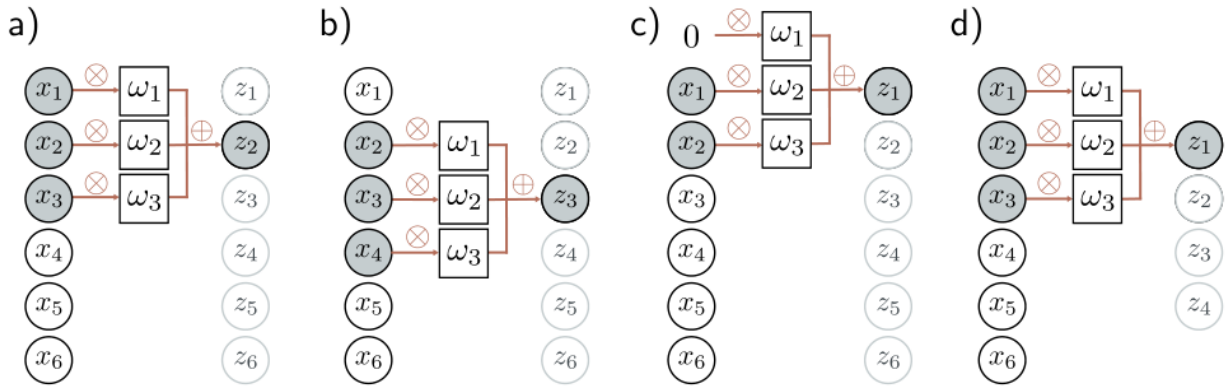


Figure 10.2 1D convolution with kernel size three. Each output z_i is a weighted sum of the nearest three inputs x_{i-1}, x_i, and x_{i+1}, where the weights are $\boldsymbol{\omega} = [\omega_1, \omega_2, \omega_3]$. a) Output z_2 is computed as $z_2 = \omega_1 x_1 + \omega_2 x_2 + \omega_3 x_3$. b) Output z_3 is computed as $z_3 = \omega_1 x_2 + \omega_2 x_3 + \omega_3 x_4$. c) At position z_1, the kernel extends beyond the first input x_1. This can be handled by zero-padding, in which we assume values outside the input are zero. The final output is treated similarly. d) Alternatively, we could only compute outputs where the kernel fits within the input range ("valid" convolution); now, the output will be smaller than the input.

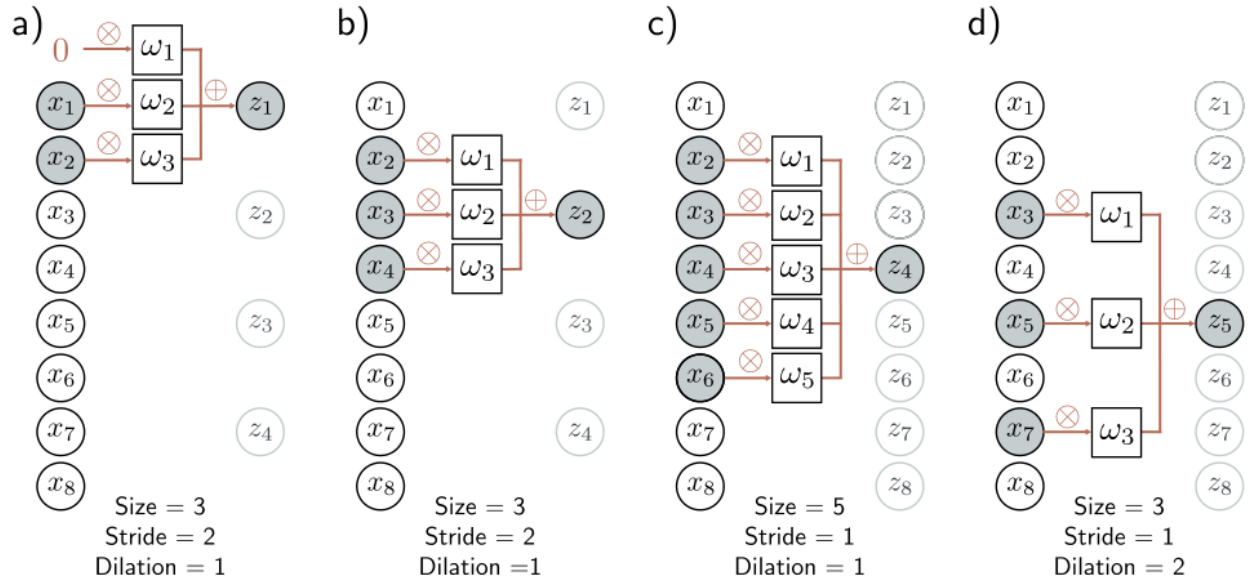


Figure 10.3 Stride, kernel size, and dilation. a) With a stride of two, we evaluate the kernel at every other position, so the first output z_1 is computed from a weighted sum centered at x_1, and b) the second output z_2 is computed from a weighted sum centered at x_3 and so on. c) The kernel size can also be changed. With a kernel size of five, we take a weighted sum of the nearest five inputs. d) In dilated or atrous convolution (from the French "à trous" – with holes), we intersperse zeros in the weight vector to allow us to combine information over a large area using fewer weights.

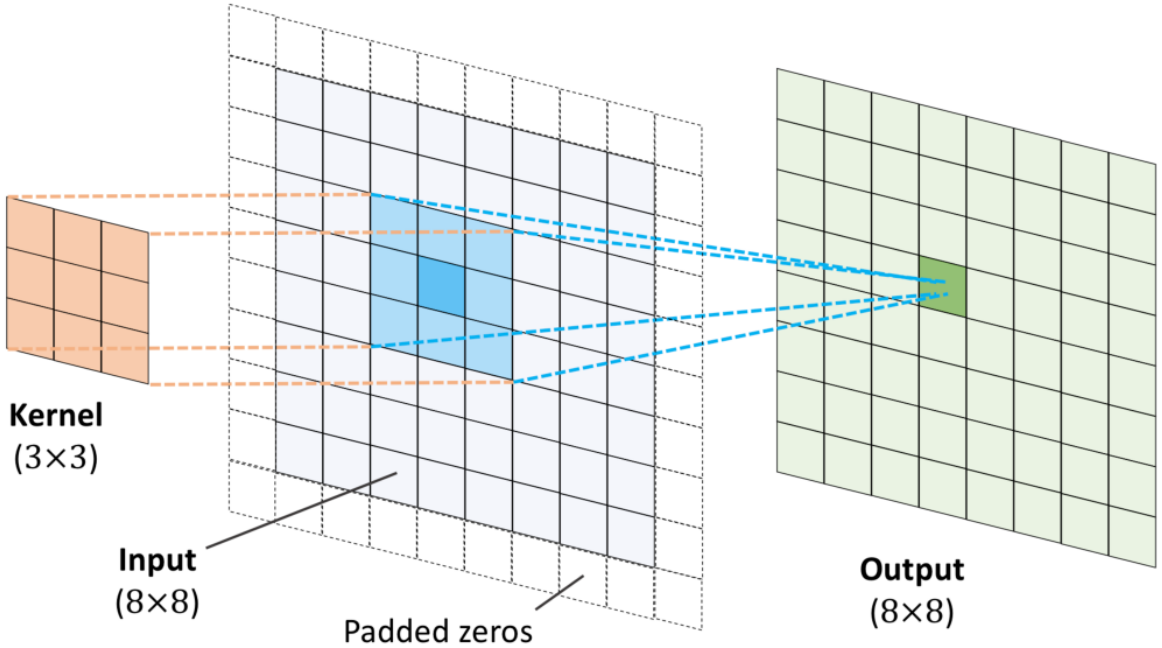


Figure 14.5: The output of a 2D convolution

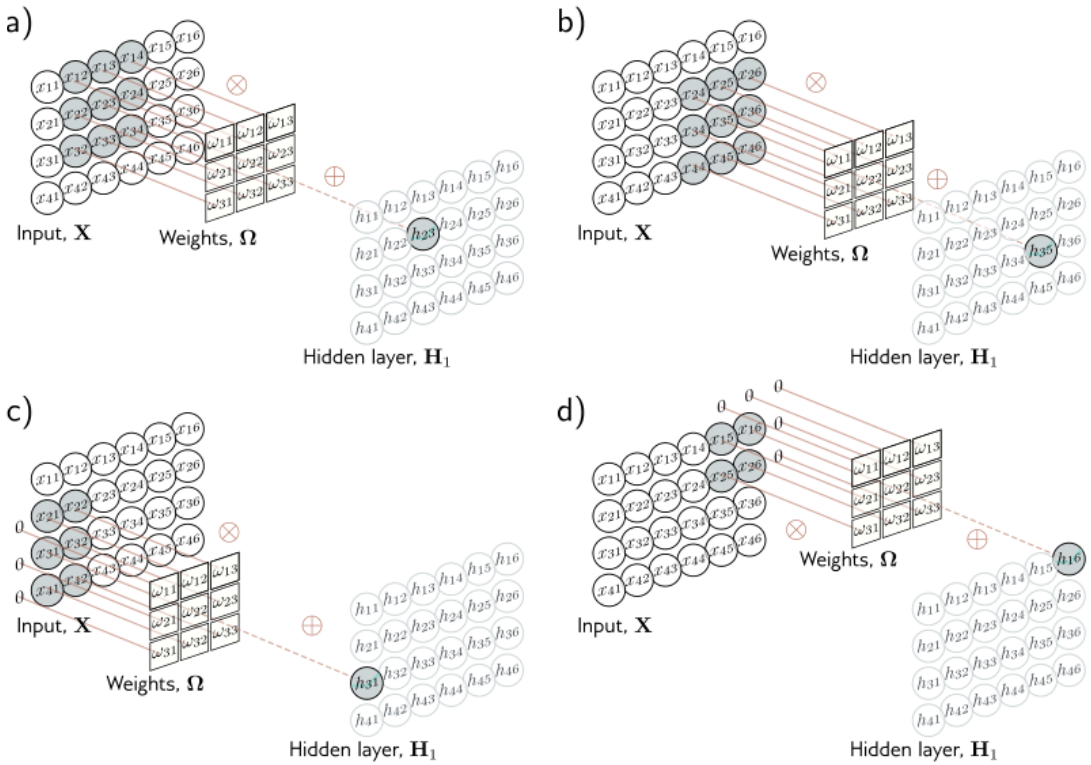


Figure 10.9 2D convolutional layer. Each output h_{ij} computes a weighted sum of the 3×3 nearest inputs, adds a bias, and passes the result through an activation function. a) Here, the output h_{23} (shaded output) is a weighted sum of the nine positions from x_{12} to x_{34} (shaded inputs). b) Different outputs are computed by translating the kernel across the image grid in two dimensions. c–d) With zero-padding, positions beyond the image's edge are considered to be zero.

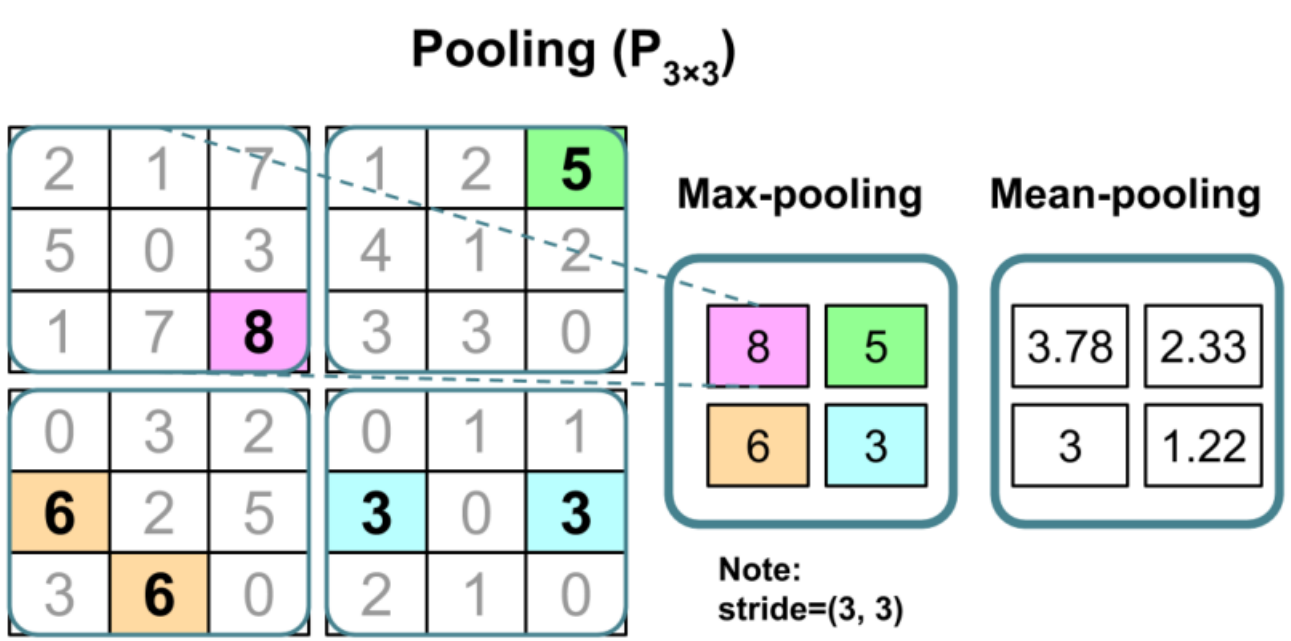


Figure 14.8: An example of max-pooling and mean-pooling

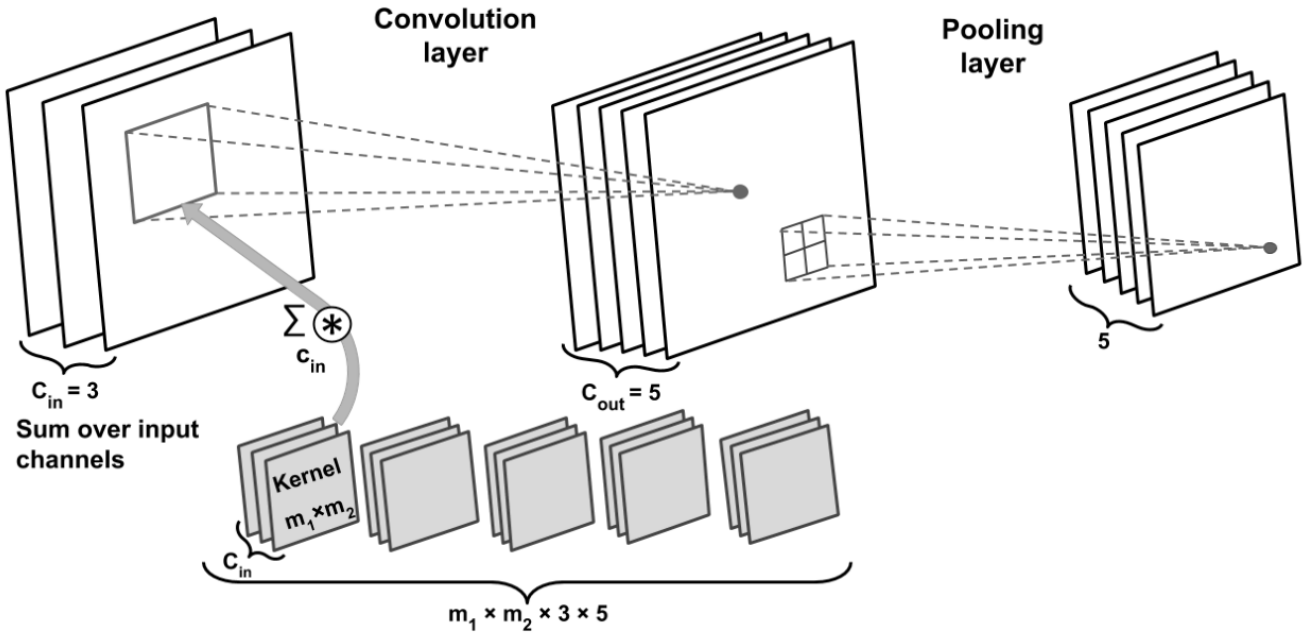
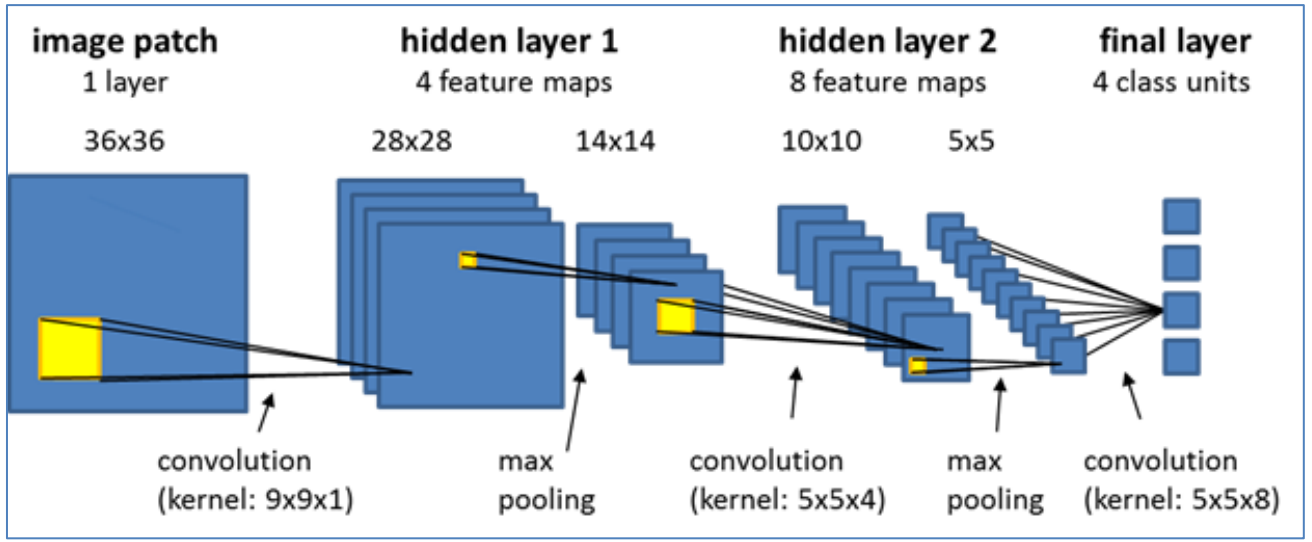


Figure 14.9: Implementing a CNN

Example: CNN

- Convolution: imagine a sliding window that moves across pixels
- Main application is computer vision, need model to match symmetries of our problem. For a picture of a bird, the bird can be anywhere in the picture, so a convolution gives a direct way to handle images
- Pooling: reduce dimension by combining inputs from previous layer



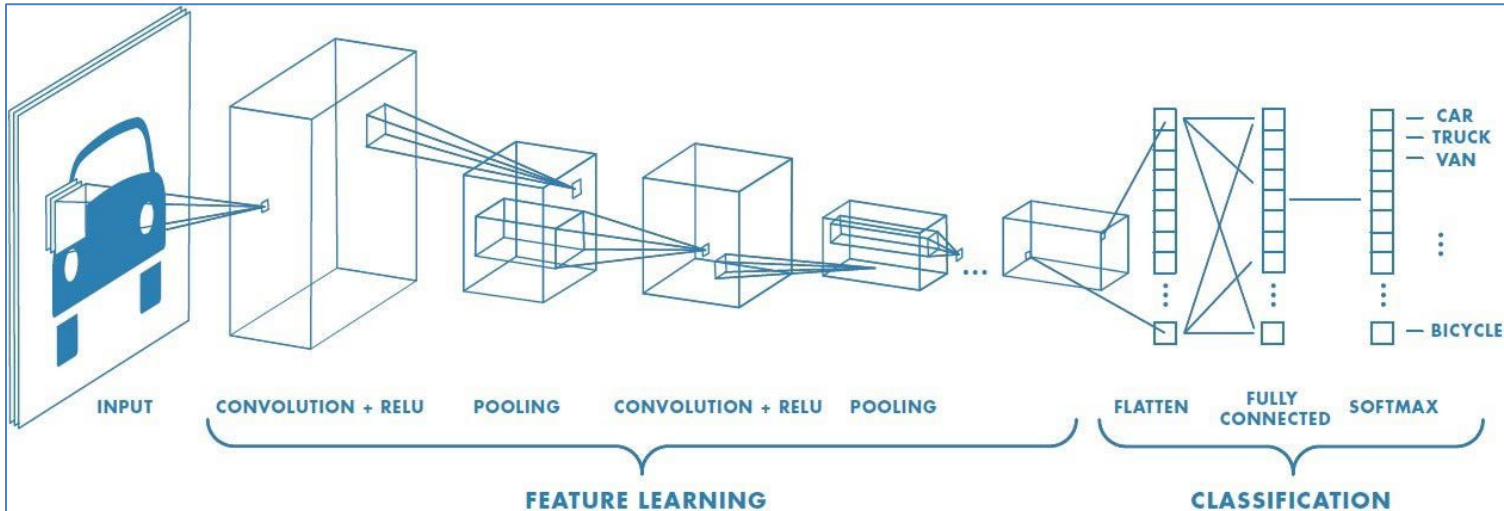
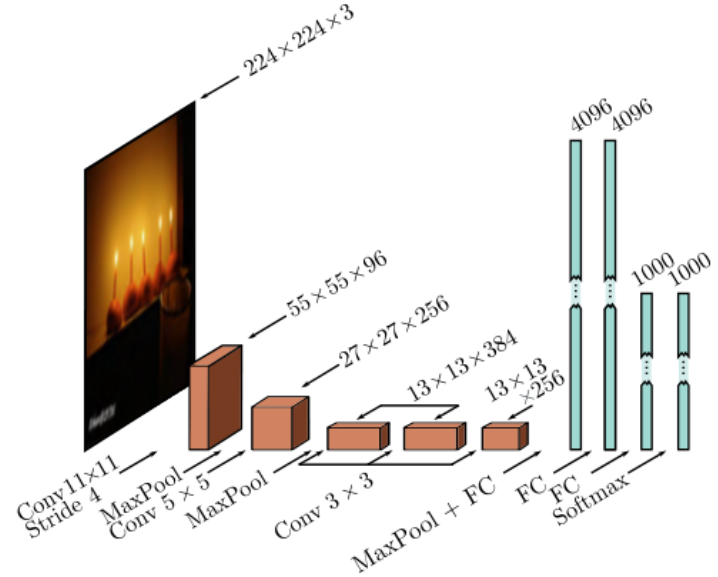
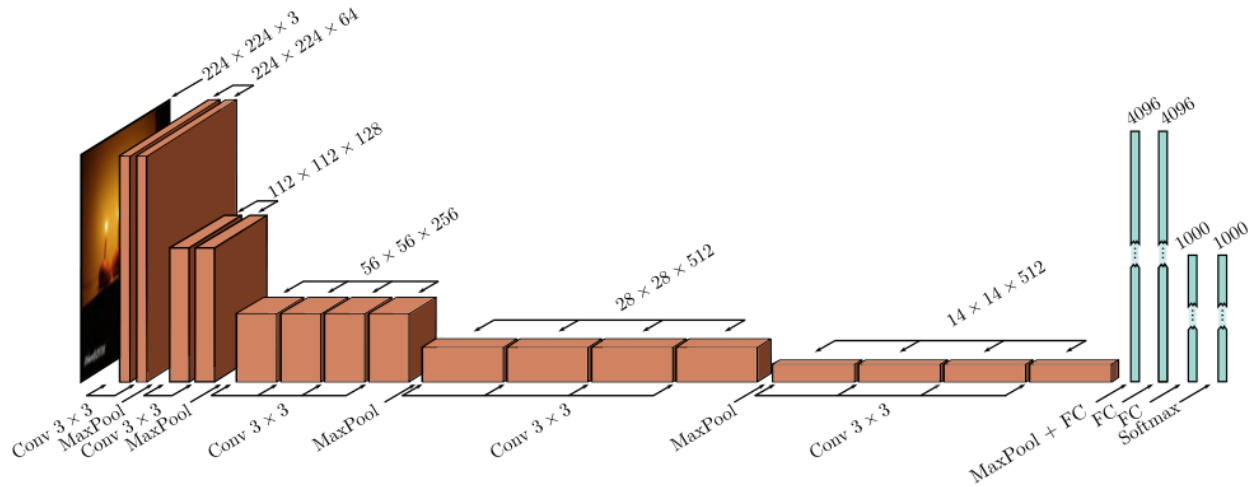


Figure 10.16 AlexNet (Krizhevsky et al., 2012). The network maps a 224×224 color image to a 1000-dimensional vector representing class probabilities. The network first convolves with 11×11 kernels and stride 4 to create 96 channels. It decreases the resolution again using a max pool operation and applies a 5×5 convolutional layer. Another max pooling layer follows, and three 3×3 convolutional layers are applied. After a final max pooling operation, the result is vectorized and passed through three fully connected (FC) layers and finally the softmax layer.



60M parameters



114M parameters

Figure 10.17 VGG network (Simonyan & Zisserman, 2014) depicted at the same scale as AlexNet (see figure 10.16). This network consists of a series of convolutional layers and max pooling operations, in which the spatial scale of the representation gradually decreases, but the number of channels gradually increases. The hidden layer after the last convolutional operation is resized to a 1D vector and three fully connected layers follow. The network outputs 1000 activations corresponding to the class labels that are passed through a softmax function to create class probabilities.

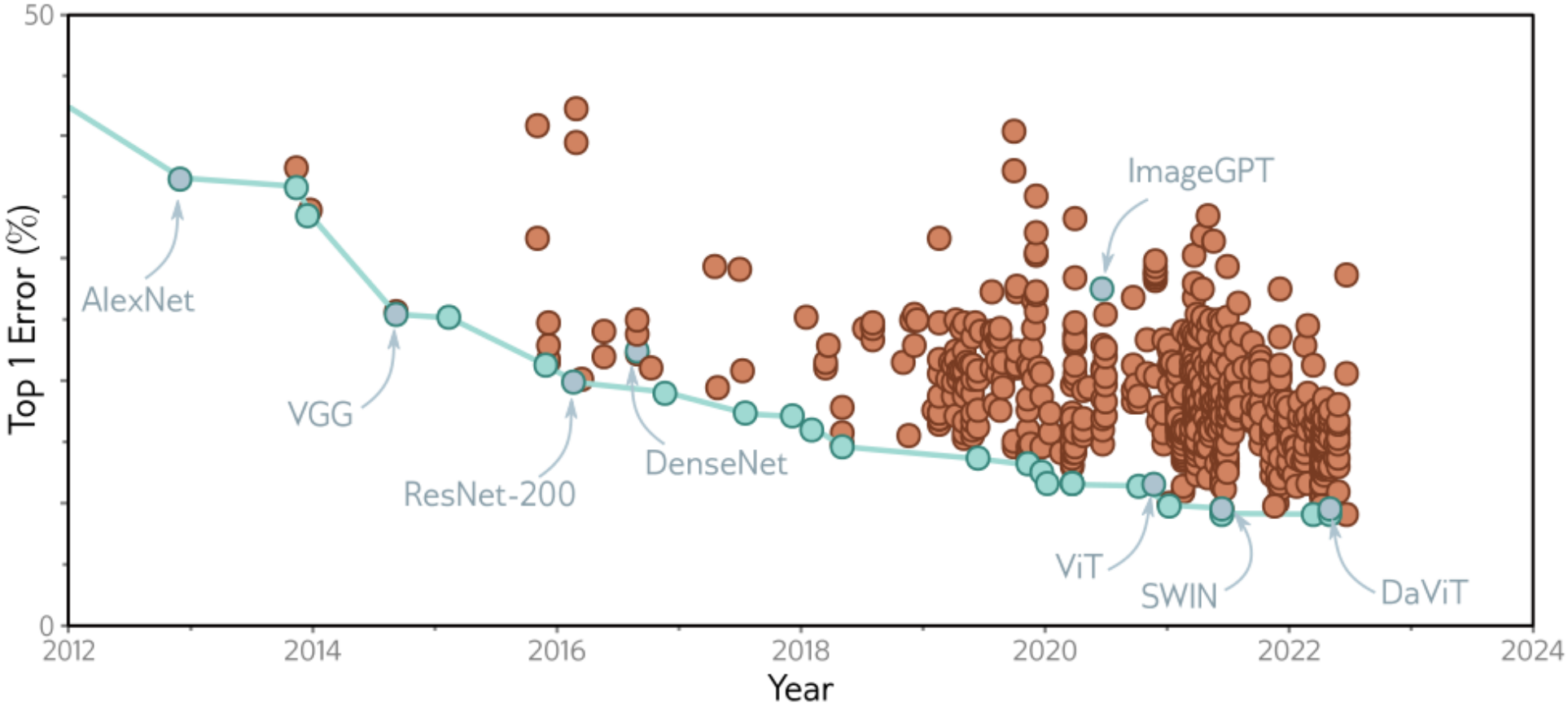
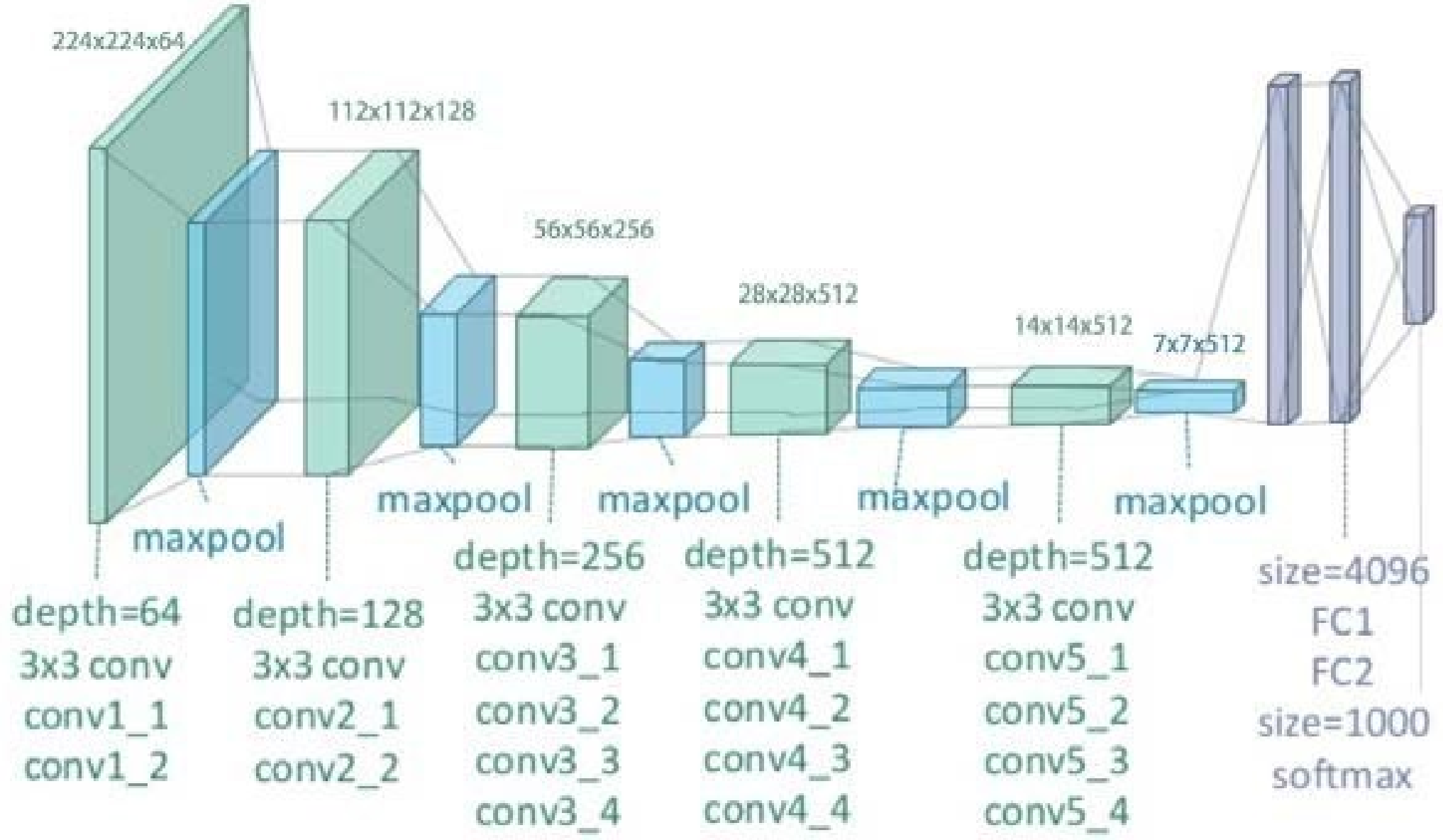


Figure 10.21 ImageNet performance. Each circle represents a different published model. Blue circles represent models that were state-of-the-art. Models discussed in this book are also highlighted. The AlexNet and VGG networks were remarkable for their time but are now far from state of the art. ResNet-200 and DenseNet are discussed in chapter 11. ImageGPT, ViT, SWIN, and DaViT are discussed in chapter 12. Adapted from https://paperswithcode.com/sota/image-classification-on-imagenet.

https://colab.research.google.com/github/tensorflow/models/blob/master/research/nst_blogpost/4_Neural_Style_Transfer_with_Eager_Execution.ipynb

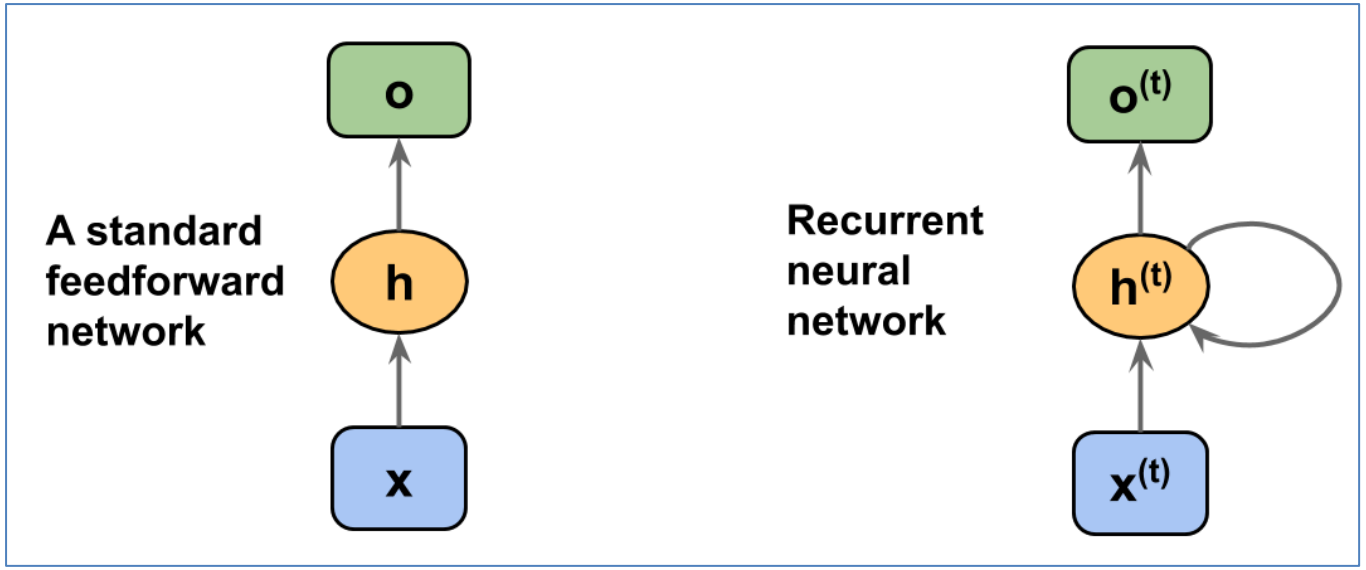


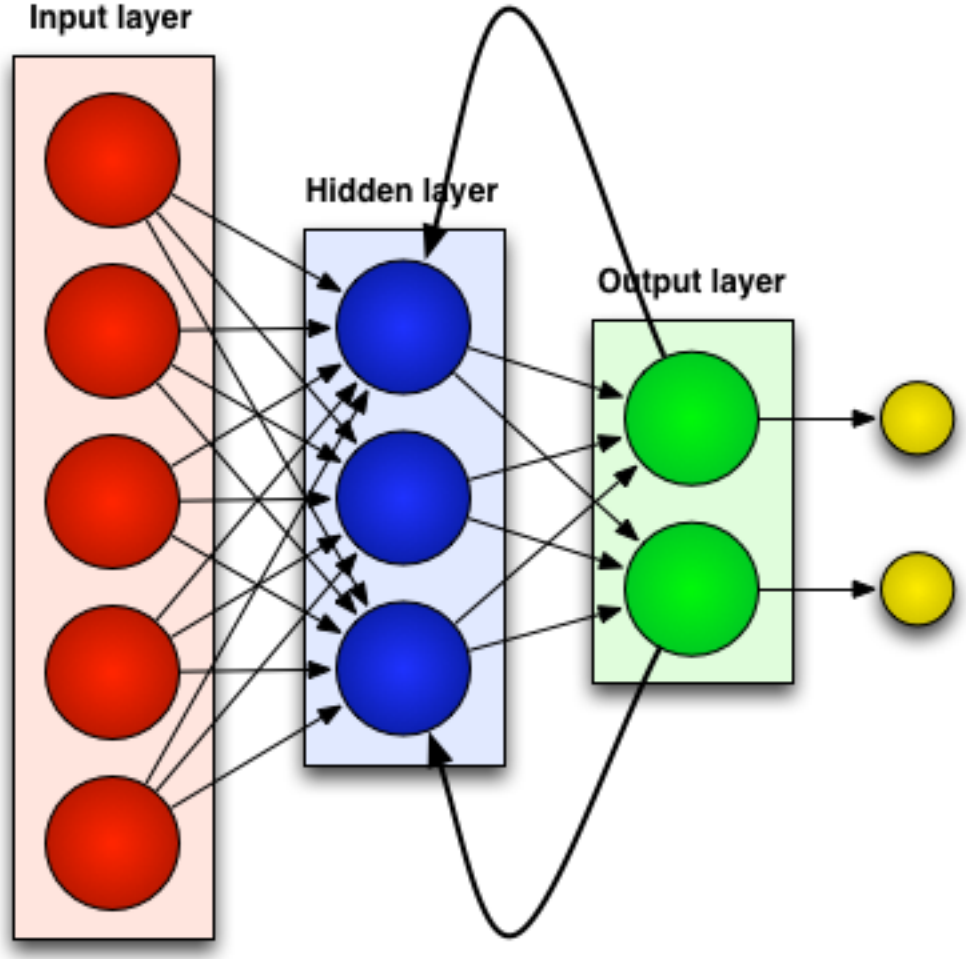
```python
class Cnn(nn.Module):
    def __init__(self, dropout=0.5):
        super(Cnn, self).__init__()
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3)
        self.conv2_drop = nn.Dropout2d(p=dropout)
        self.fc1 = nn.Linear(1600, 100) # 1600 = number channels * width * height
        self.fc2 = nn.Linear(100, 10)
        self.fc1_drop = nn.Dropout(p=dropout)

    def forward(self, x):
        x = torch.relu(F.max_pool2d(self.conv1(x), 2))
        x = torch.relu(F.max_pool2d(self.conv2_drop(self.conv2(x)), 2))

        # flatten over channel, height and width = 1600
        x = x.view(-1, x.size(1) * x.size(2) * x.size(3))

        x = torch.relu(self.fc1_drop(self.fc1(x)))
        x = torch.softmax(self.fc2(x), dim=-1)
        return x
```

https://github.com/skorch-dev/skorch/blob/master/notebooks/MNIST.ipynb

Recurrent Neural Network

- Adds limited “short term memory” to NN
- Allows for context in classifiers
- https://en.wikipedia.org/wiki/Recurrent_neural_network





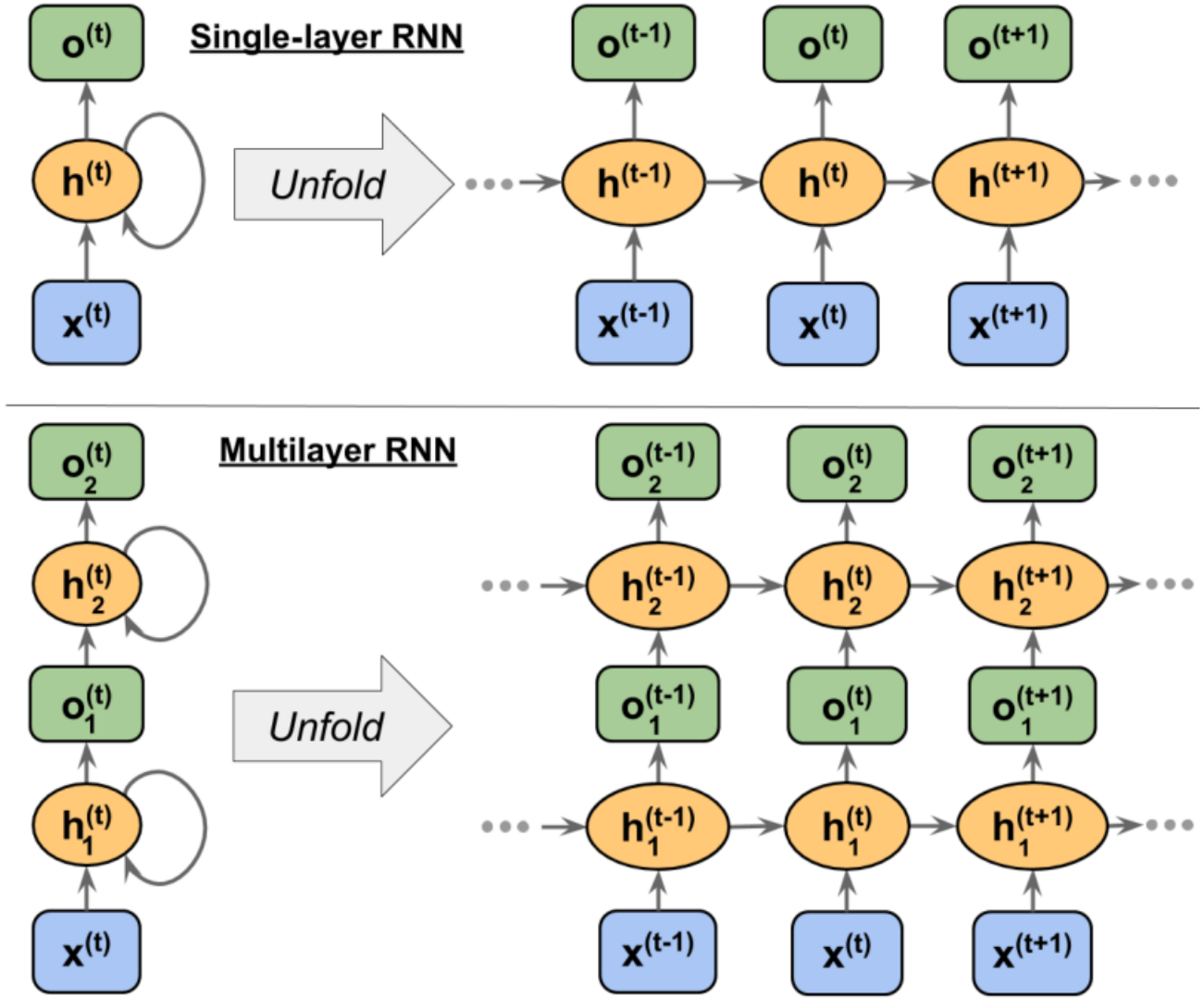


Figure 15.4: Examples of an RNN with one and two hidden layers

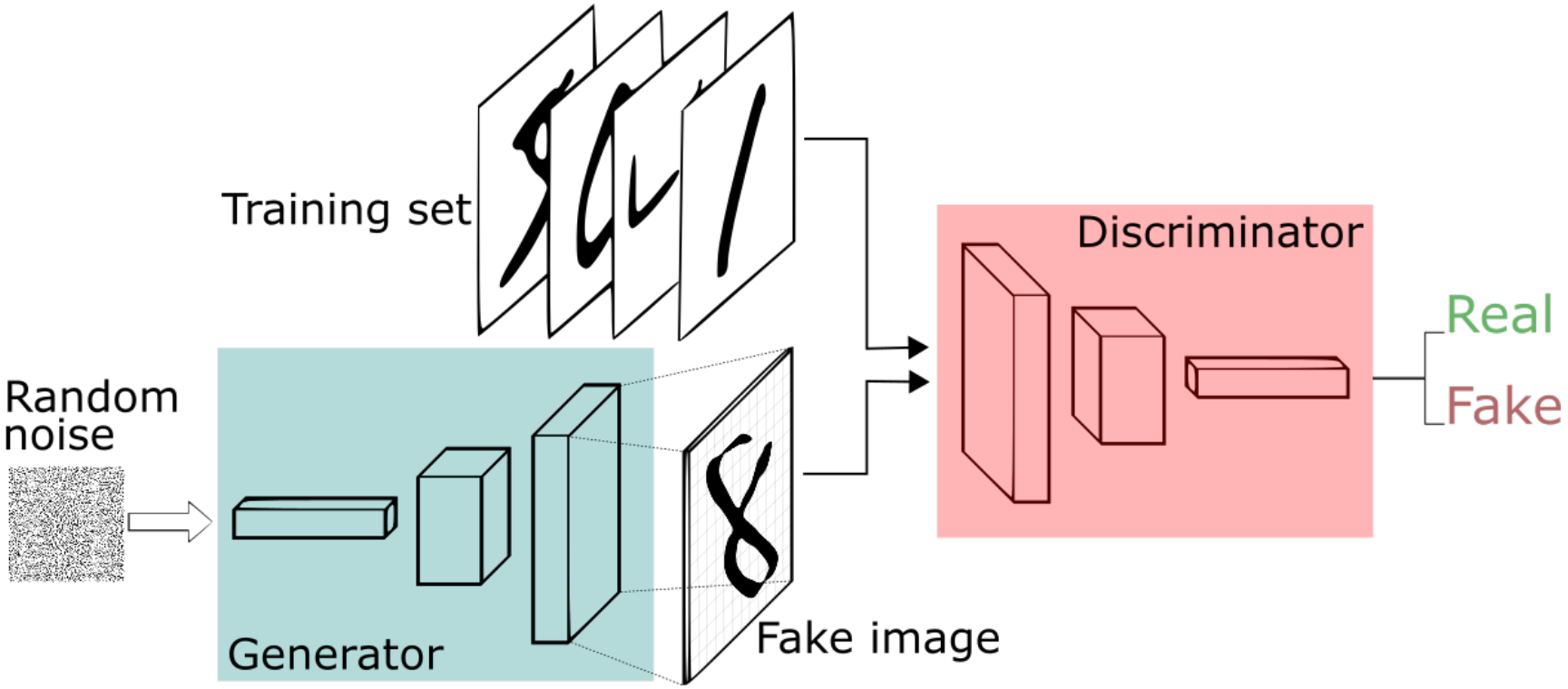
GANS

https://arxiv.org/abs/1710.10196

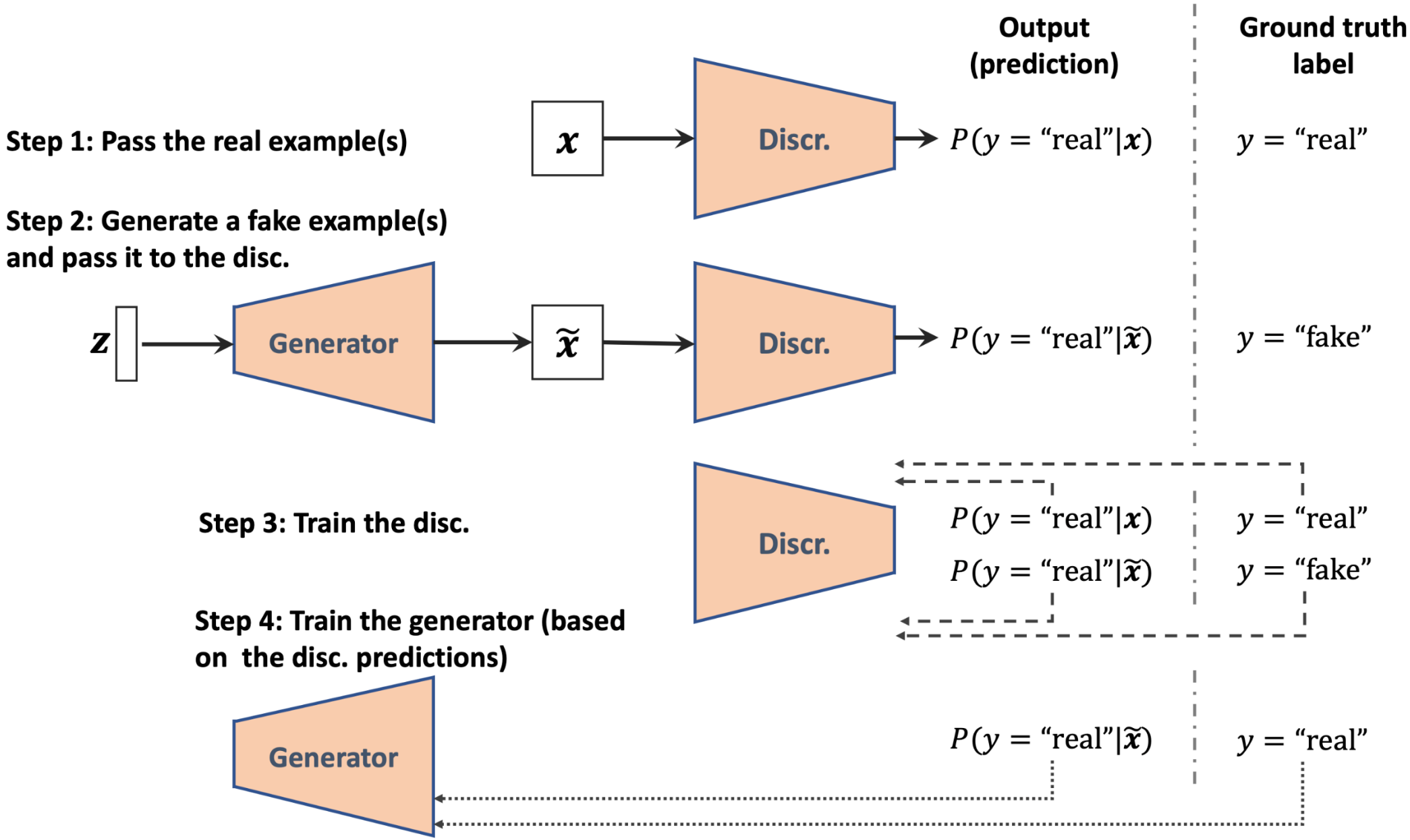
NONE OF THESE PEOPLE ARE REAL

Generative Adversarial Networks

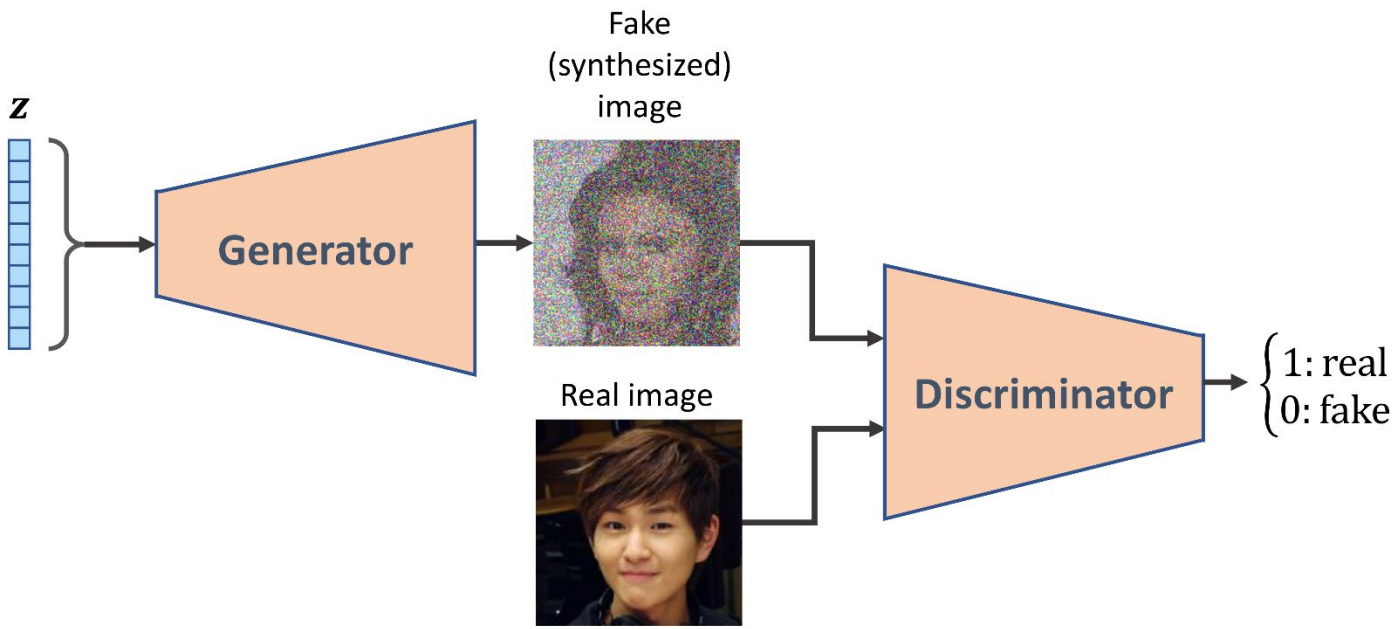
- Very clever trick to generate realistic data
- Proposed in 2014 - https://arxiv.org/abs/1406.2661
- Create a generator and a discriminator that learn from each other and improve together

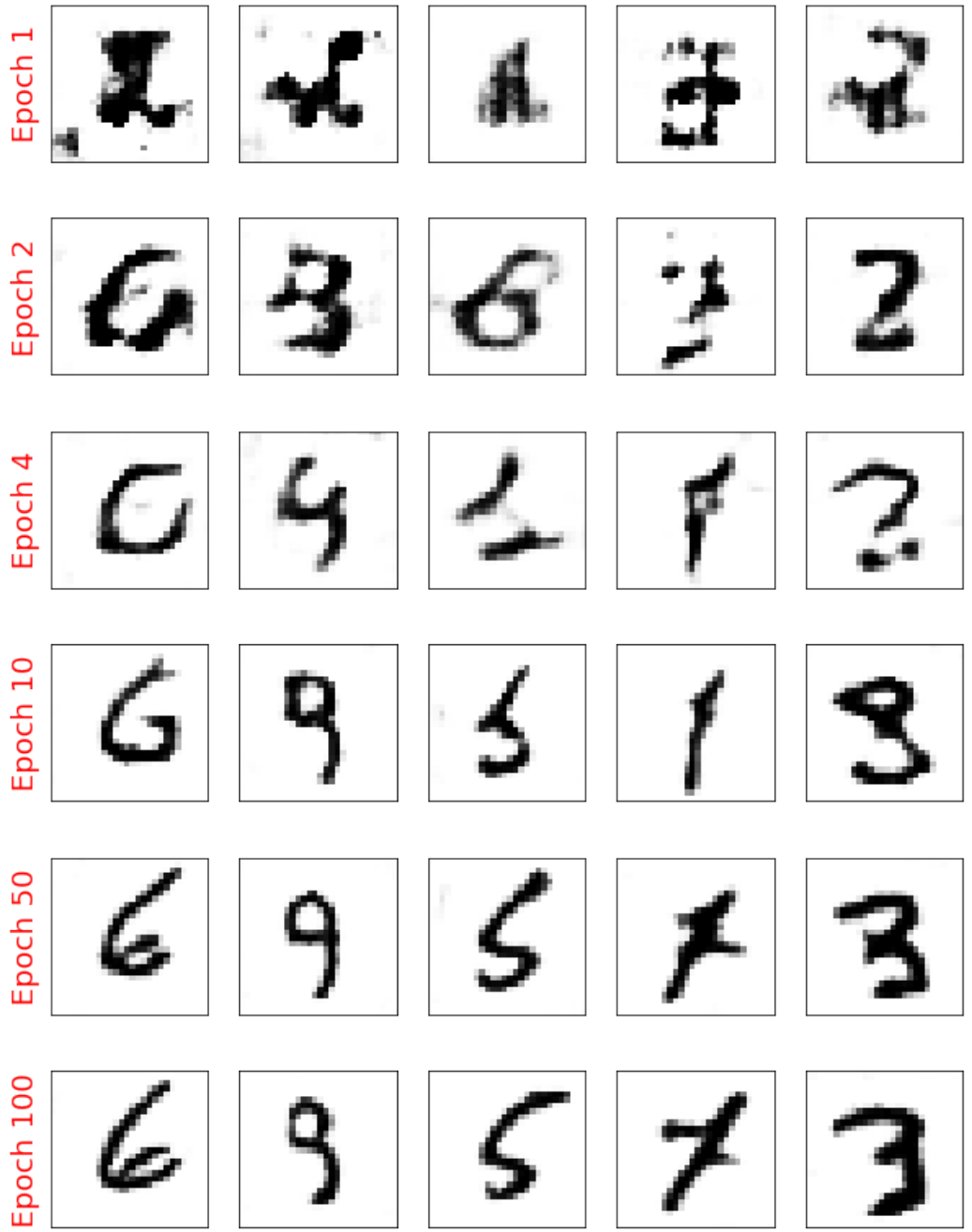


https://github.com/rasbt/machine-learning-book/blob/main/ch17/ch17_part1.ipynb



https://github.com/rasbt/machine-learning-book/blob/main/ch17/ch17_part1.ipynb





Deep Convolutional GAN

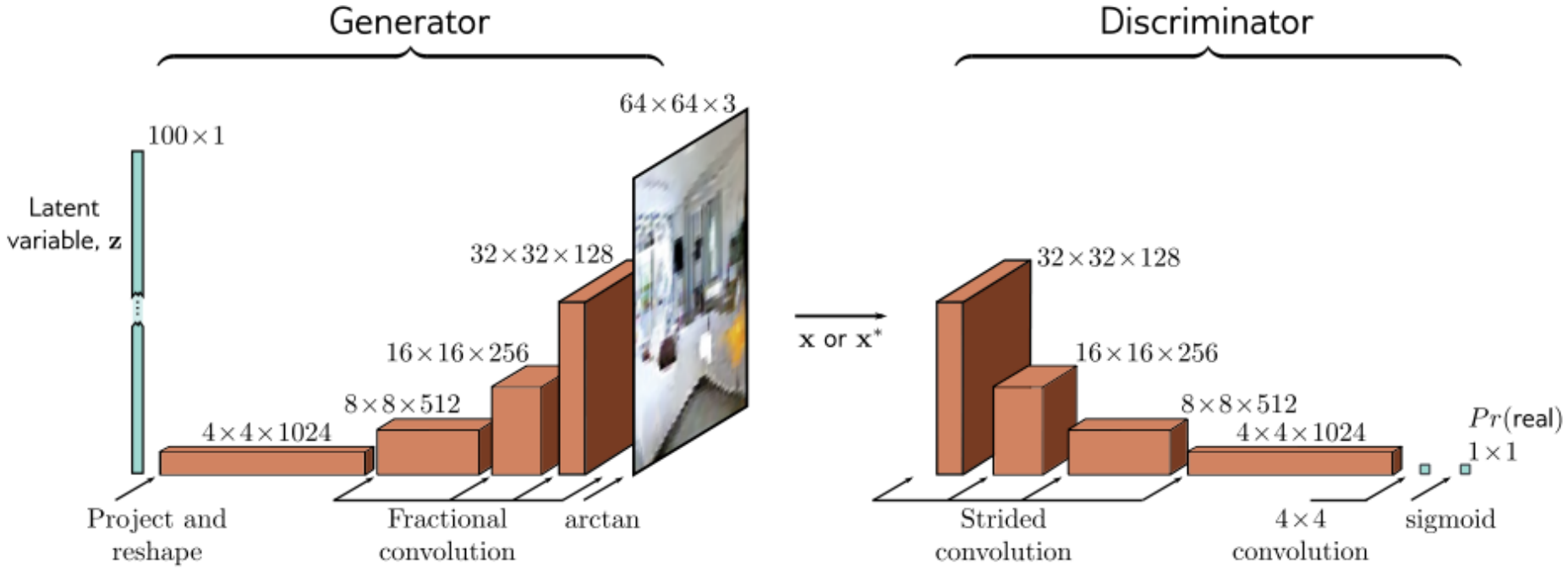


Figure 15.3 DCGAN architecture. In the generator, a 100D latent variable $\mathbf{z}$ is drawn from a uniform distribution and mapped by a linear transformation to a 4×4 representation with 1024 channels. This is then passed through a series of convolutional layers that gradually upsample the representation and decrease the number of channels. At the end is an arctan function that maps the 64×64×3 representation to a fixed range so that it can represent an image. The discriminator consists of a standard convolutional net that classifies the input as either a real example or a generated sample.

Generative Adversarial Networks

Resources

- https://arxiv.org/abs/1406.2661
- https://en.wikipedia.org/wiki/Generative_adversarial_network
- https://machinelearningmastery.com/what-are-generative-adversarial-networks-gans/
- https://github.com/rasbt/machine-learning-book/blob/main/ch17/ch17_part1.ipynb

Transformers

Transformers

More examples of these later. Goal is to connect different representations:

- encoders – transform input (text, image, speech) into generic representation
- decoder – predict next token of text/sound/image
- encoder-decoder – translate different languages or inputs

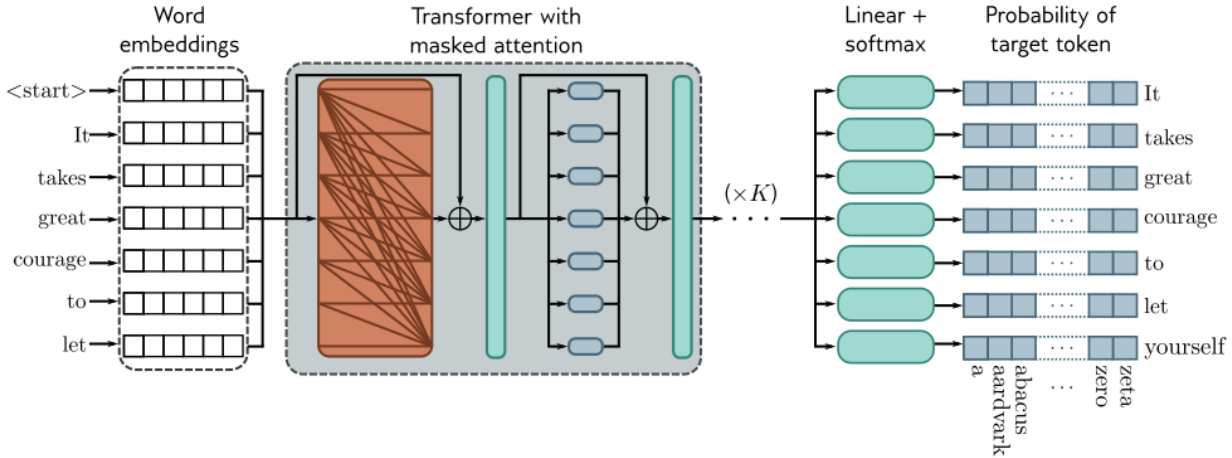
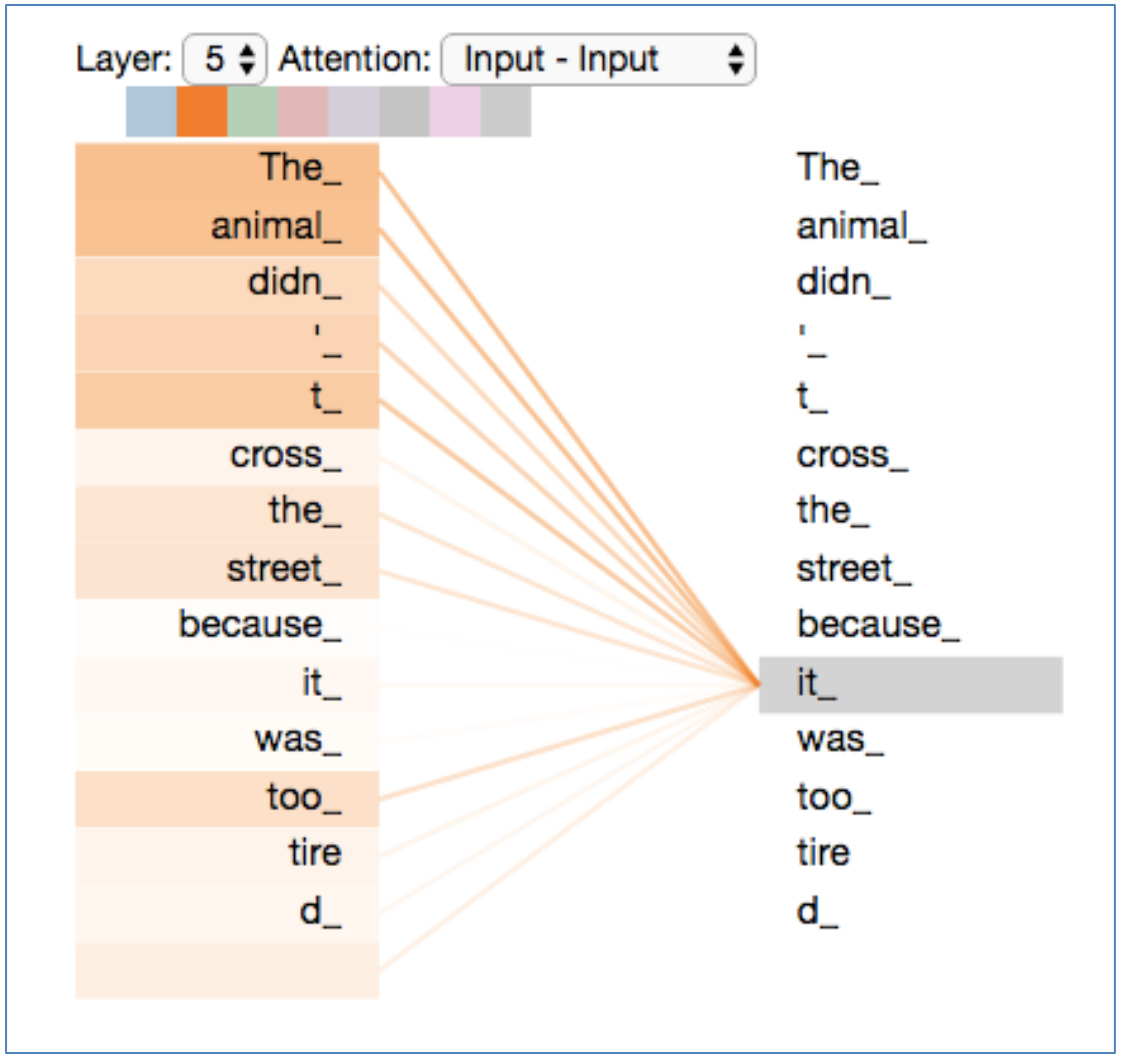
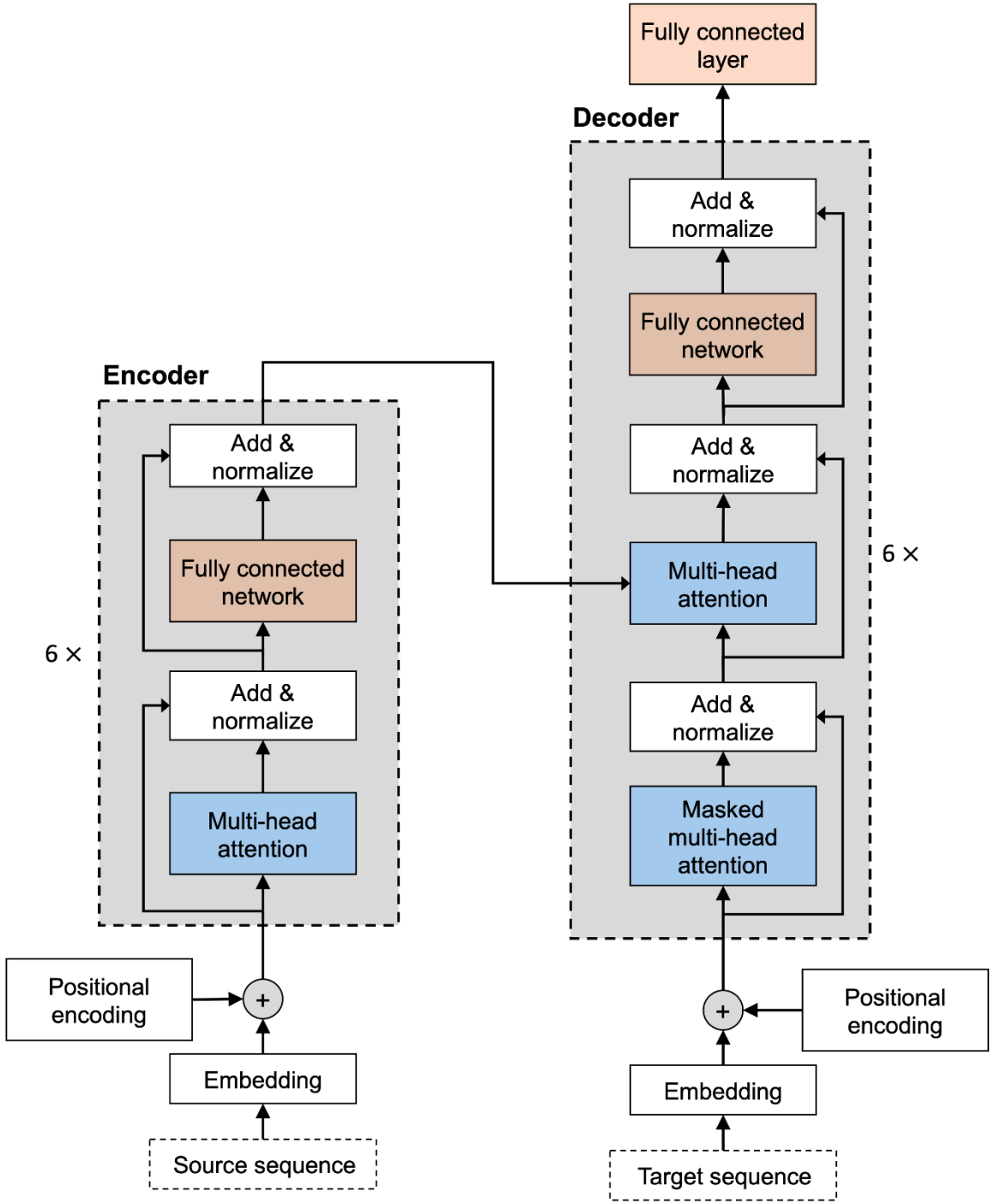


Figure 12.12 Training GPT3-type decoder network. The tokens are mapped to word embeddings with a special <start> token at the beginning of the sequence. The embeddings are passed through a series of transformer layers that use masked self-attention. Here, each position in the sentence can only attend to its own embedding and those of tokens earlier in the sequence (orange connections). The goal at each position is to maximize the probability of the following ground truth token in the sequence. In other words, at position one, we want to maximize the probability of the token It; at position two, we want to maximize the probability of the token takes; and so on. Masked self-attention ensures the system cannot cheat by looking at subsequent inputs. The autoregressive task has the advantage of making efficient use of the data since every word contributes a term to the loss function. However, it only exploits the left context of each word.



http://jalammar.github.io/illustrated-transformer/



Reinforcement Learning

Reinforcement

- we have an agent that can take actions
- establish reward
- learn how to choose actions that lead to high rewards

Examples:

- learn to make a robot walk
- social media algorithm
- learn to maximize score on a game

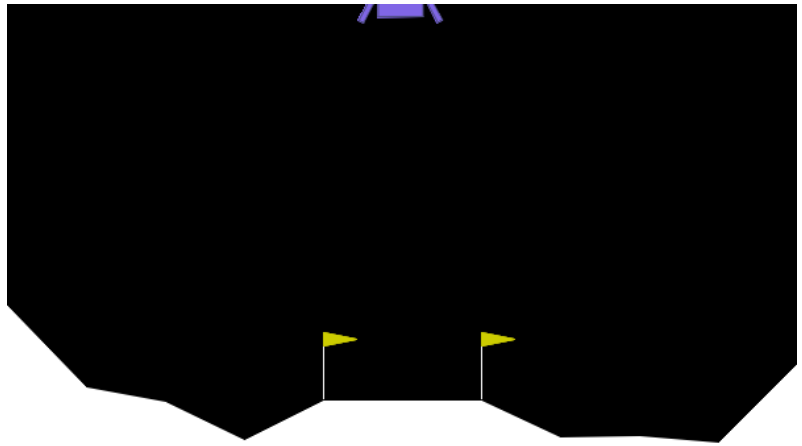
Reinforcement

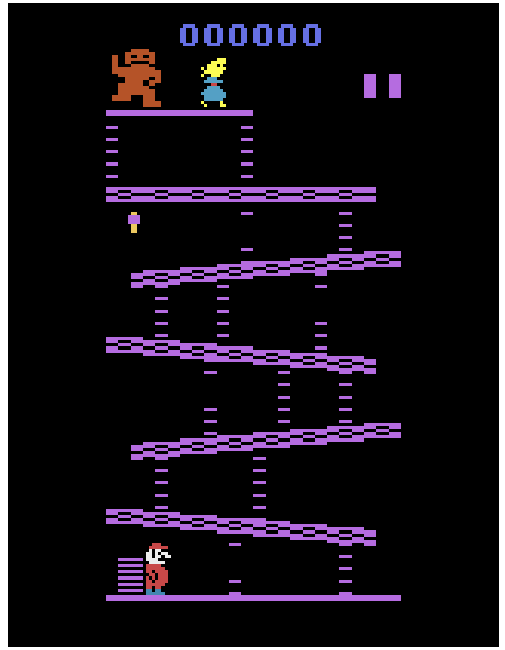
- Never tell the agent HOW to behave directly, instead we specify a reward that it learns to achieve
- REINFORCEMENT = starts with random behavior, we reward good actions
- These slides are only going to introduce the topic, some vocabulary, a some basic algorithms. This is a big topic and we will only scratch the surface

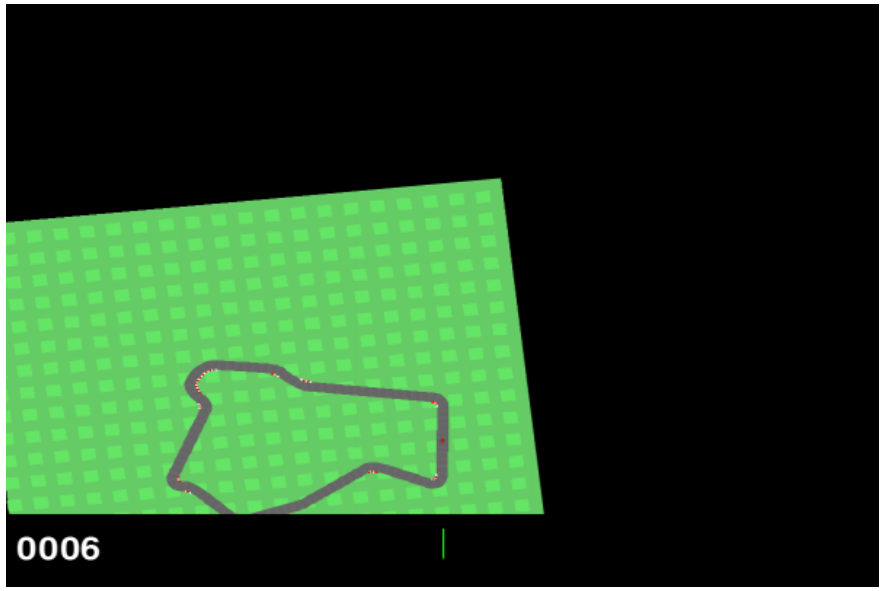
Challenges:

- reward can be delayed from action - winning or losing a game after many turns)
- environment can be random – opponent or conditions can change so that same action yields different results
- agents must balance exploration with scoring (exploration-exploitation tradeoff)

https://gymnasium.farama.org/



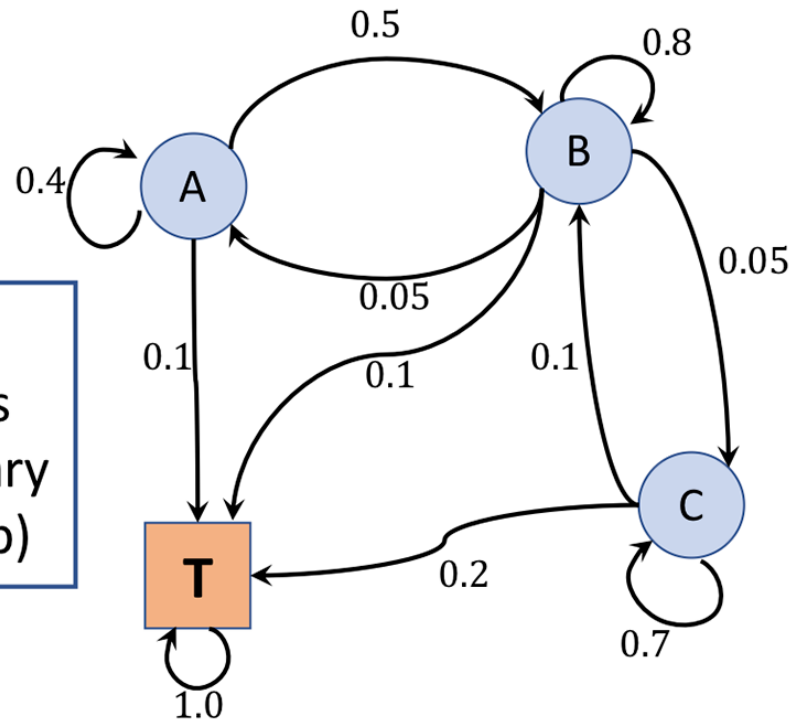




Markov Process

Exist in one of a finite set of states, each state has probability to transition to other states

A: studying at home
B: playing video games
C: studying at the library
T: terminal state (sleep)



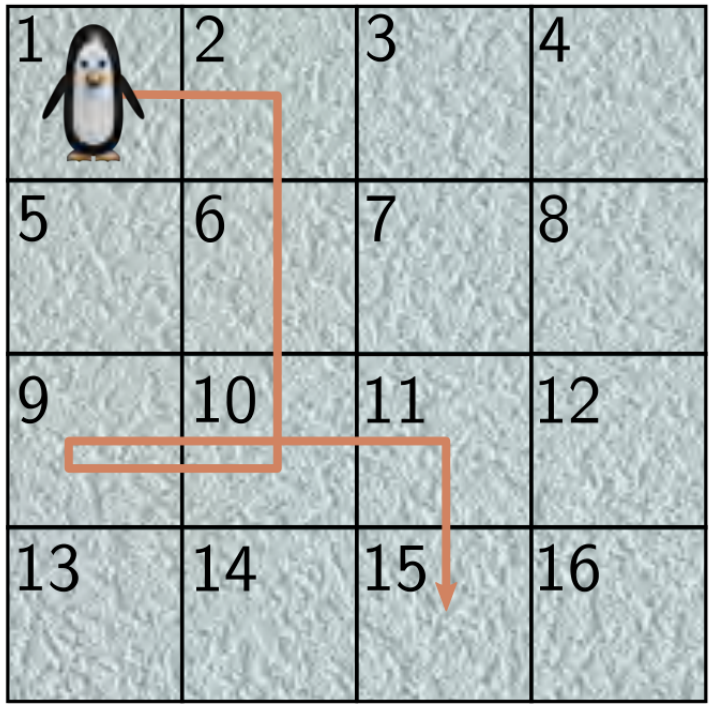
Transition probabilities:

	A	*B*	*C*	*T*
A	**0.4**	**0.5**	**0.0**	**0.1**
B	**0.05**	**0.8**	**0.05**	**0.1**
C	**0.0**	**0.1**	**0.7**	**0.2**
T	**0.0**	**0.0**	**0.0**	**1.0**

Markov Process

Exist in one of a finite set of states, each state has probability to transition to other states

a)



$s_1, s_2, s_3, s_4, s_5, s_6, s_7, s_8$

$\boldsymbol{\tau} = [1, 2, 6, 10, 9, 10, 11, 15]$

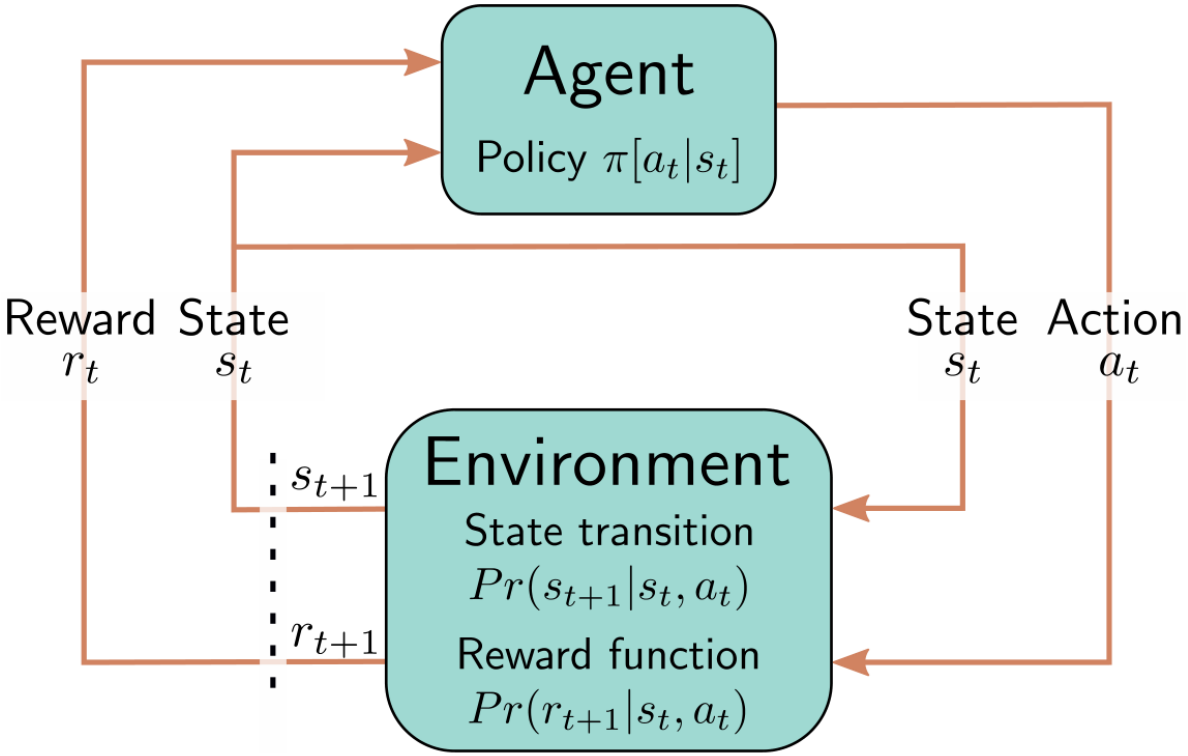
b)

$$
\begin{array}{c}
s_t \\
\begin{matrix} s_{t+1} \end{matrix}
\end{array}
\begin{bmatrix}
0 & 0.33 & 0 & 0 & 0.33 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0.5 & 0 & 0.33 & 0 & 0 & 0.25 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0.33 & 0 & 0.5 & 0 & 0 & 0.25 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0.33 & 0 & 0 & 0 & 0 & 0.33 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0.5 & 0 & 0 & 0 & 0 & 0.25 & 0 & 0 & 0.33 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0.33 & 0 & 0 & 0.33 & 0 & 0.25 & 0 & 0 & 0.25 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0.33 & 0 & 0 & 0.25 & 0 & 0.33 & 0 & 0 & 0.25 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0.5 & 0 & 0 & 0.25 & 0 & 0 & 0 & 0 & 0.33 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0.33 & 0 & 0 & 0 & 0 & 0.25 & 0 & 0 & 0.5 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0.25 & 0 & 0 & 0.33 & 0 & 0.25 & 0 & 0 & 0.33 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0.25 & 0 & 0 & 0.25 & 0 & 0.33 & 0 & 0 & 0.33 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0.33 & 0 & 0 & 0.25 & 0 & 0 & 0 & 0 & 0.5 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0.33 & 0 & 0 & 0 & 0 & 0.33 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0.25 & 0 & 0 & 0.5 & 0 & 0.33 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0.25 & 0 & 0 & 0.33 & 0 & 0.5 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0.33 & 0 & 0 & 0.33 & 0
\end{bmatrix}
$$

$$Pr(s_{t+1}|s_t)$$

Vocab

- our **agent** takes **actions** in an **environment** to earn **rewards**
- episode – sequence of actions
- s_t = state at time t
- a_t = action at time t
- π = policy, rules for choosing action
- r_t = reward



notation assumes rewards at next time step, so a_t gets r_{t+1}

To help us find the optimal policy we will use:

G_t = return = sum of future rewards with discount factor $\gamma \in (0,1]$

$$G_t \stackrel{\text{def}}{=} R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0} \gamma^k R_{t+k+1}$$

v = state value = expected return from a state

$$v[s_t|\pi] = \mathbb{E}\Big[G_t|s_t, \pi\Big]$$

q = action value = expected return from action in a state

$$q[s_t, a_t|\pi] = \mathbb{E}\Big[G_t|s_t, a_t, \pi\Big]$$

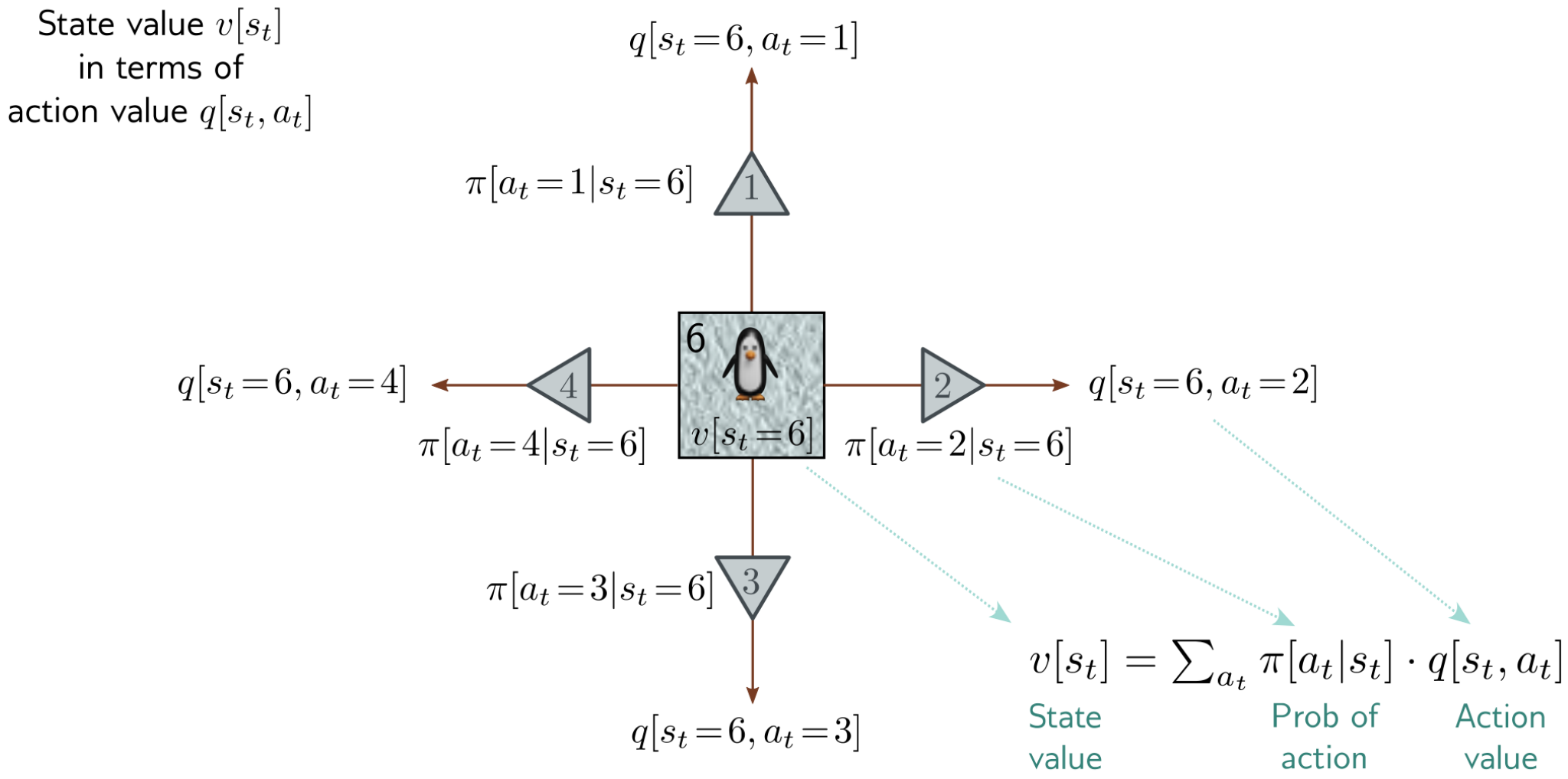
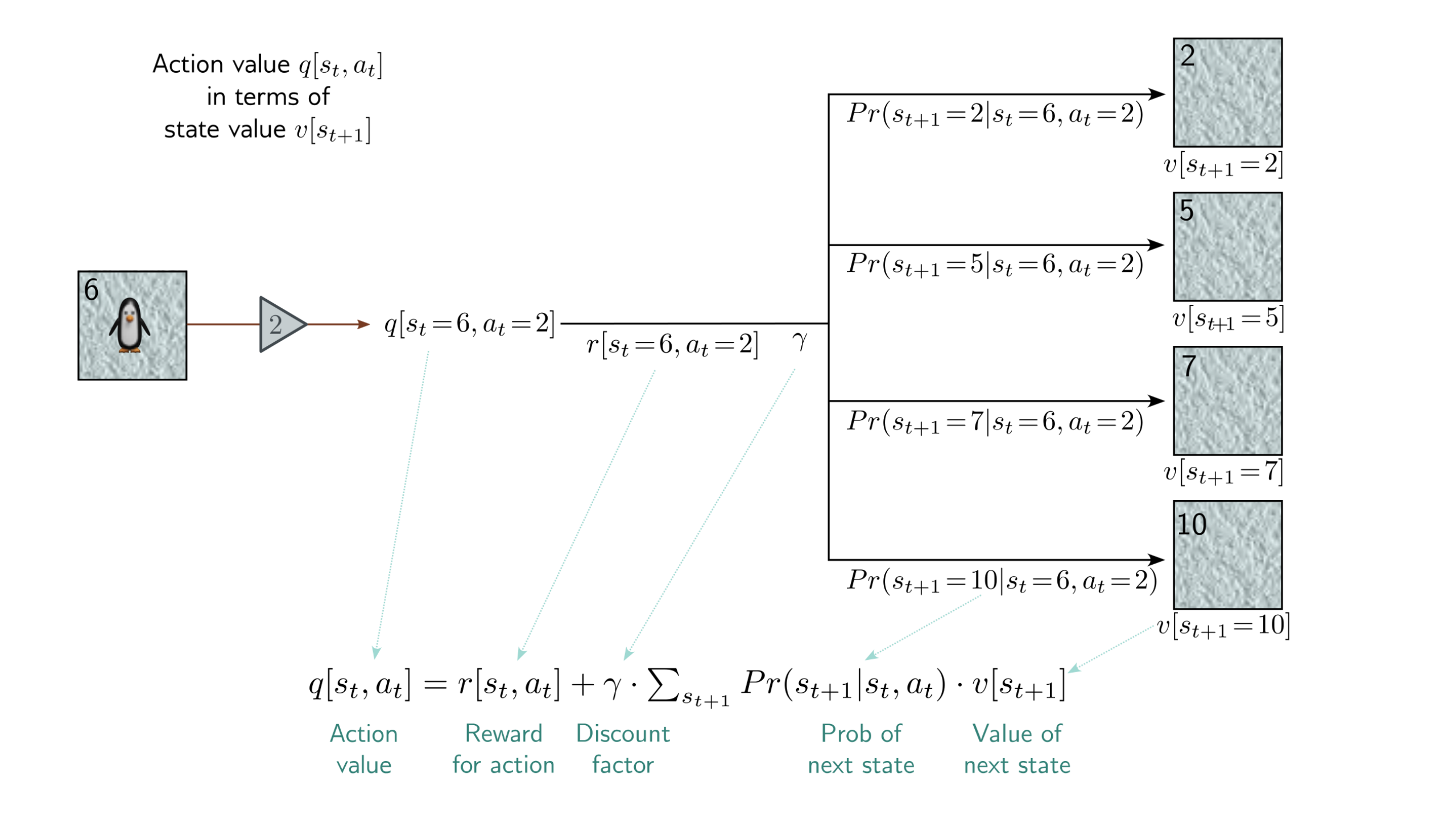


Figure 19.8 Relationship between state values and action values. The value of state six $v[s_t=6]$ is a weighted sum of the action values $q[s_t=6, a_t]$ at state six, where the weights are the policy probabilities $\pi[a_t|s_t=6]$ of taking that action.


Action value $q[s_t, a_t]$
in terms of
state value $v[s_{t+1}]$
6
2
$q[s_t = 6, a_t = 2]$
$r[s_t = 6, a_t = 2]$
γ
$Pr(s_{t+1} = 2 | s_t = 6, a_t = 2)$
$Pr(s_{t+1} = 5 | s_t = 6, a_t = 2)$
$Pr(s_{t+1} = 7 | s_t = 6, a_t = 2)$
$Pr(s_{t+1} = 10 | s_t = 6, a_t = 2)$
2
5
7
10
$v[s_{t+1} = 2]$
$v[s_{t+1} = 5]$
$v[s_{t+1} = 7]$
$v[s_{t+1} = 10]$
$q[s_t, a_t] = r[s_t, a_t] + \gamma \cdot \sum_{s_{t+1}} Pr(s_{t+1} | s_t, a_t) \cdot v[s_{t+1}]$
Action value
Reward for action
Discount factor
Prob of next state
Value of next state

Bellman Equations (1950's)

$$v[s_t] = \sum_{a_t} \pi[a_t|s_t] q[s_t, a_t]$$

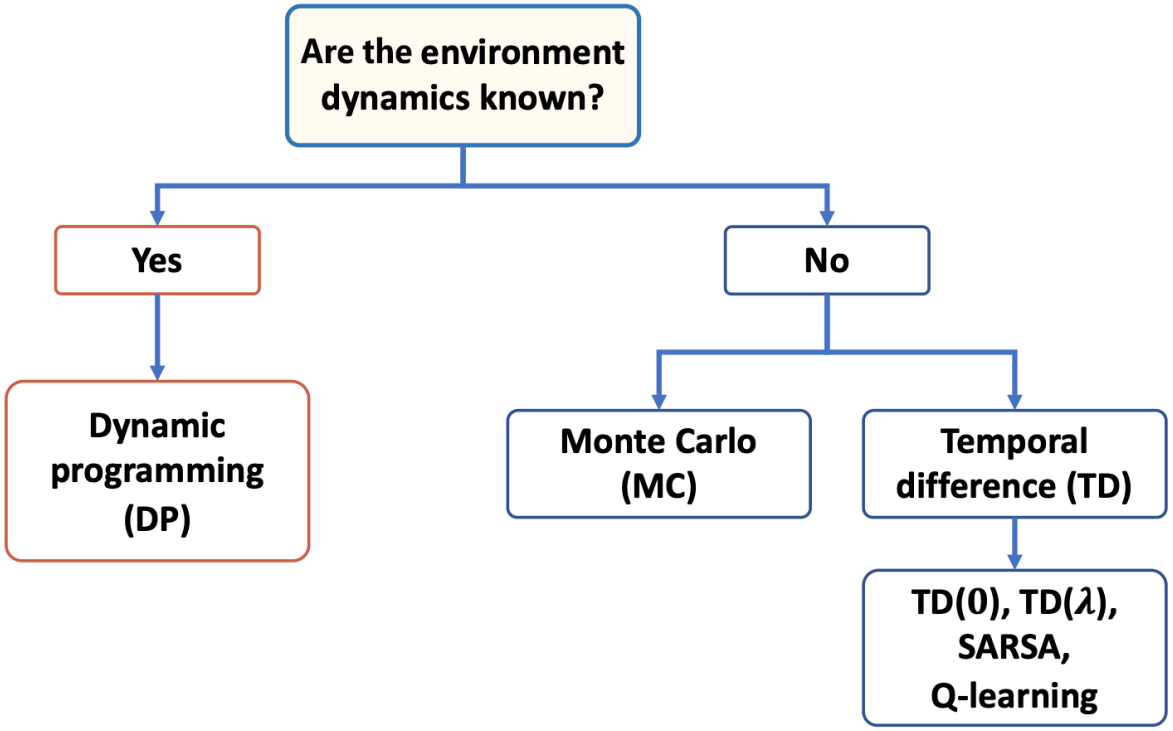
$$q[s_t, a_t] = r[s_t, a_t] + \gamma \cdot \sum_{s_{t+1}} Pr(s_{t+1}|s_t, a_t) v[s_{t+1}]$$

insert two previous equations into each other

$$v[s_t] = \sum_{a_t} \pi[a_t|s_t] \left(r[s_t, a_t] + \gamma \cdot \sum_{s_{t+1}} Pr(s_{t+1}|s_t, a_t) v[s_{t+1}] \right)$$

$$q[s_t, a_t] = r[s_t, a_t] + \gamma \cdot \sum_{s_{t+1}} Pr(s_{t+1}|s_t, a_t) \left(\sum_{a_{t+1}} \pi[a_{t+1}|s_{t+1}] q[s_{t+1}, a_{t+1}] \right)$$

RL Algorithms



On-policy: we use our best policy while learning

Off-policy: we can take other actions for more exploration

Q-Learning

The optimal policy is always to choose the action with the highest q value, so if we know all of the q's then our problem is solved

$$\pi[a_t|s_t] \leftarrow \underset{a_t}{\operatorname{argmax}}\left[q^*[s_t, a_t]\right]$$

$$G_t \stackrel{\text{def}}{=} R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0} \gamma^k R_{t+k+1}$$

$$G_t = R_{t+1} + \gamma G_{t+1} = r + \gamma G_{t+1}$$

$$\begin{aligned} q_\pi(s,a) &\stackrel{\text{def}}{=} E_\pi[G_t|S_t = s, A_t = a] \\ &= E_\pi[r + \gamma G_{t+1}|S_t = s, A_t = a] \\ &= r + \gamma\, E_\pi[G_{t+1}|S_t = s, A_t = a] \end{aligned}$$

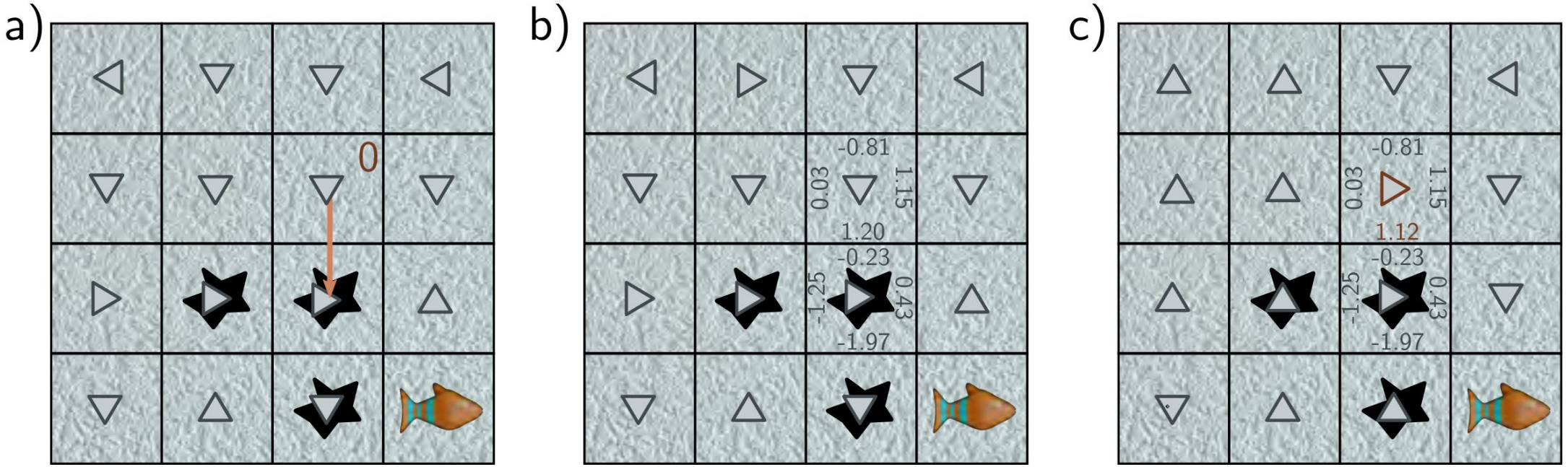
Q-Learning

$$q[s_t, a_t] = r[s_t, a_t] + \gamma \cdot \sum_{s_{t+1}} Pr(s_{t+1}|s_t, a_t) \left(\sum_{a_{t+1}} \pi[a_{t+1}|s_{t+1}] q[s_{t+1}, a_{t+1}] \right)$$

Use the formula: $q_t = r + \gamma q_{t+1}$

To make learning more stable, instead of just replacing q_{old} with q_{new} we make a partial change with learning rate α: $q_{new} = q_{old} + \alpha(q_{new} - q_{old})$

$$q[s_t, a_t] \leftarrow q[s_t, a_t] + \alpha \left(r[s_t, a_t] + \gamma \cdot \max_a [q[s_{t+1}, a]] - q[s_t, a_t] \right)$$



$$q[s_t, a_t] \leftarrow q[s_t, a_t] + \alpha\Big(r[s_t, a_t] + \gamma \cdot \max_a\big[q[s_{t+1}, a]\big] - q[s_t, a_t]\Big)$$

$$1.12 \leftarrow 1.20 + 0.1\Big(0.0 + 0.9 \cdot \max\big[-0.23, 0.43, -1.97, -1.25\big] - 1.20\Big)$$

Figure 19.12 Q-learning. a) The agent starts in state s_t and takes action $a_t = 2$ according to the policy. It does not slip on the ice, so it moves downward, receiving reward $r[s_t, a_t] = 0$ for leaving the original state. b) The maximum action value at the new state is found (here 0.43). c) The action value for action 2 in the original state is updated to 1.12 based on the current estimate of the maximum action value at the subsequent state, the reward, discount factor $\gamma = 0.9$, and learning rate $\alpha = 0.1$. This changes the highest action value at the original state, so the policy changes.

https://gymnasium.farama.org/introduction/train_agent/

Gym intro does Q-learning update from scratch with Bellman

Understanding the Q-Learning Update

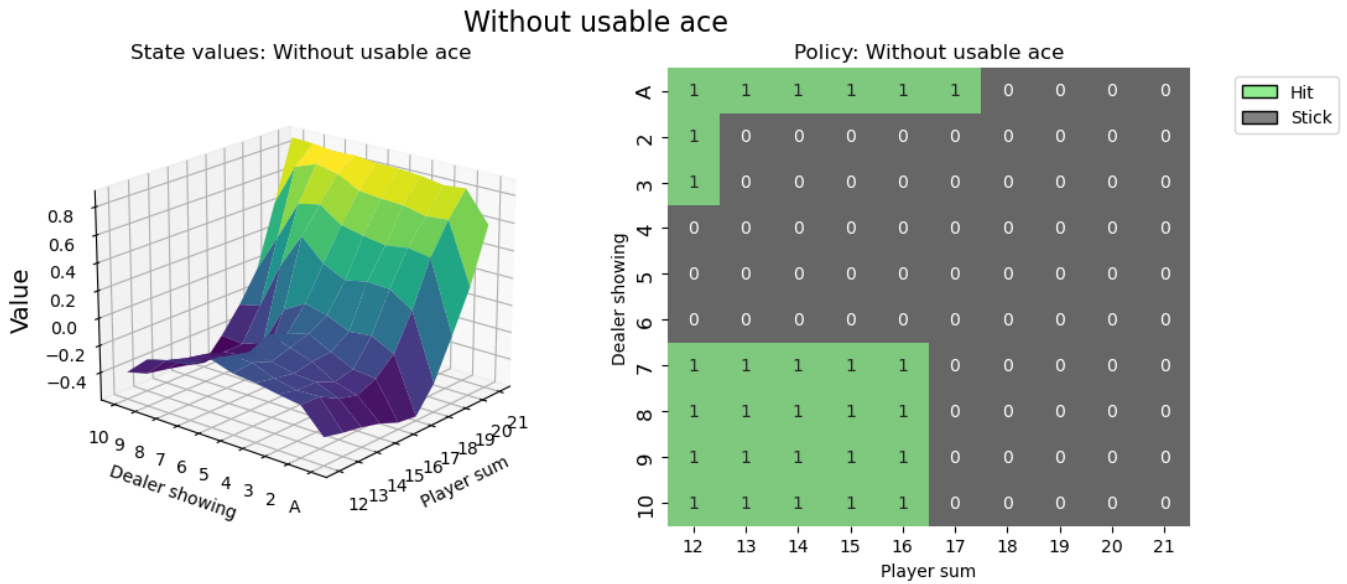
The core learning happens in the `update` method. Let's break down the math:

```
# Current estimate: Q(state, action)
current_q = self.q_values[obs][action]

# What we actually experienced: reward + discounted future value
target = reward + self.discount_factor * max(self.q_values[next_obs])

# How wrong were we?
error = target - current_q

# Update estimate: move toward the target
new_q = current_q + learning_rate * error
```

This is the famous **Bellman equation** in action - it says the value of a state-action pair should equal the immediate reward plus the discounted value of the best next action.

https://gymnasium.farama.org/tutorials/training_agents/blackjack_q_learning/#sphx-glr-tutorials-training-agents-blackjack-q-learning-py

Q-learning (model free) from scratch



Deep-Q

Q-learning works by constructing a table $q[s_t, a_t]$ for all possible states and actions. In lots of problems this is wildly impractical. Chess has about 10^{40} possible states.

Solution: replace giant q table with a neural network.

- input is a state
- one output for each possible action

Stability also improved by replay buffer that stores many old steps, and NN updates use a randomly sampled batch

$$L[\boldsymbol{\phi}] = \left(r[\mathbf{s}_t, a_t] + \gamma \cdot \max_a \left[q[\mathbf{s}_{t+1}, a, \boldsymbol{\phi}] \right] - q[\mathbf{s}_t, a_t, \boldsymbol{\phi}] \right)^2$$

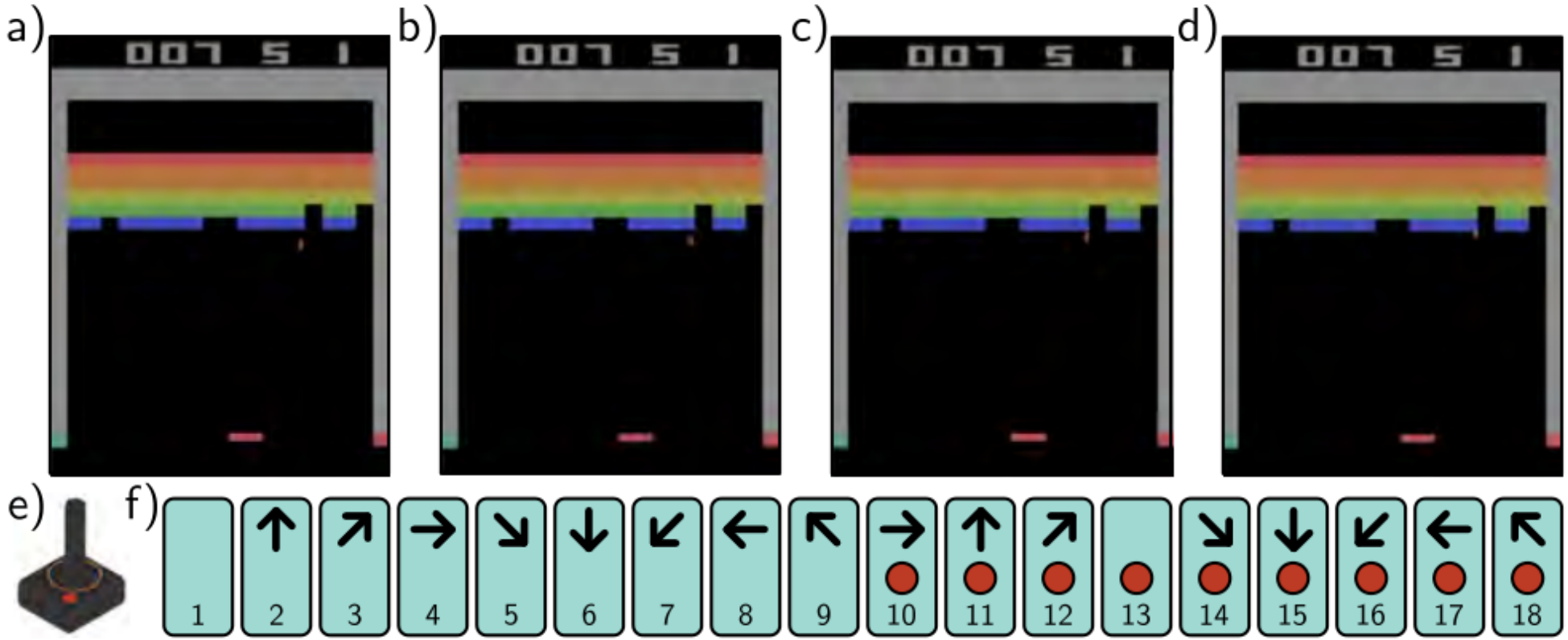
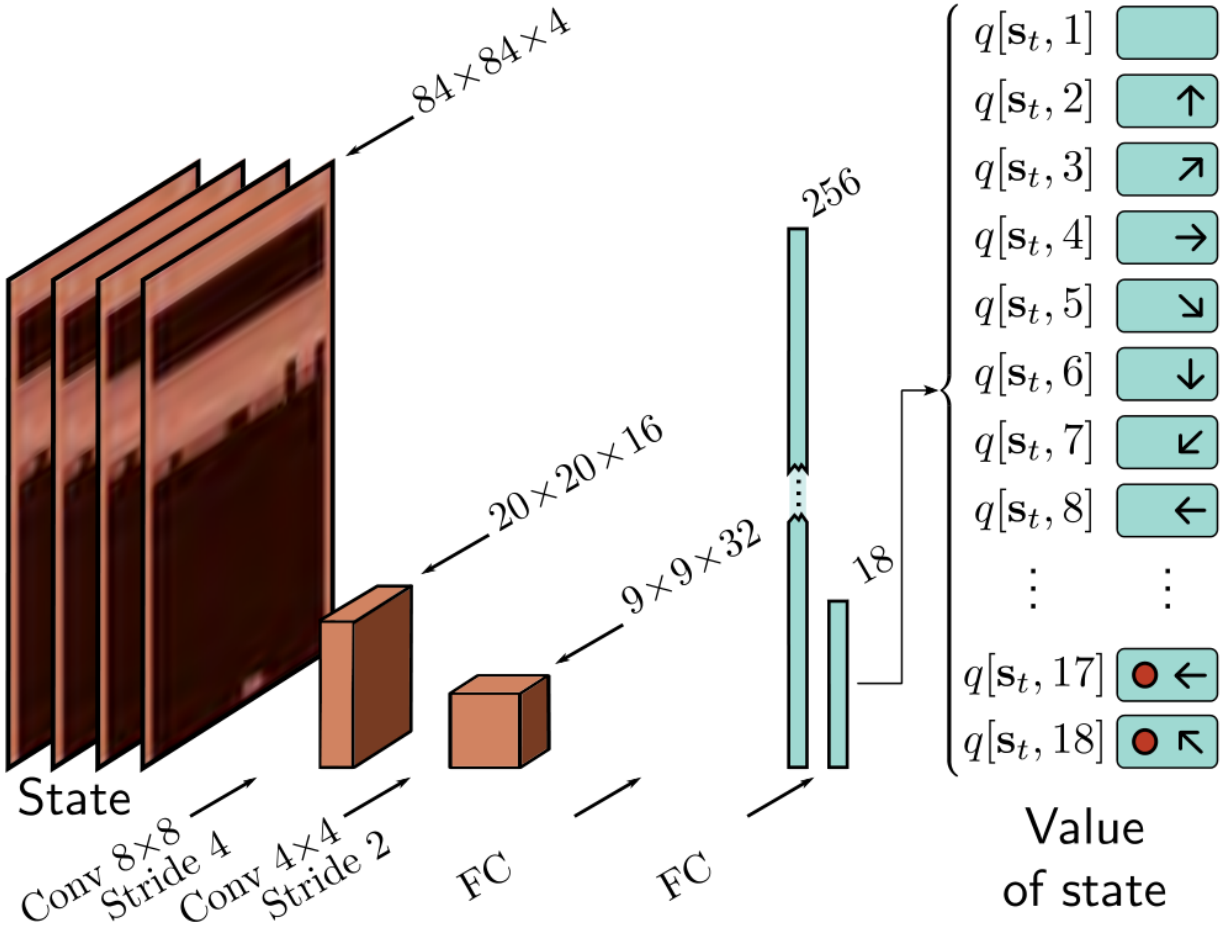


Figure 19.13 Atari Benchmark. The Atari benchmark consists of 49 Atari 2600 games, including Breakout (pictured), Pong, and various shoot-em-up, platform, and other types of games. a-d) Even for games with a single screen, the state is not fully observable from a single frame because the velocity of the objects is unknown. Consequently, it is usual to use several adjacent frames (here, four) to represent the state. e) The action simulates the user input via a joystick. f) There are eighteen actions corresponding to eight directions of movement or no movement, and for each of these nine cases, the button being pressed or not.

Figure 19.14 Deep Q-network architecture. The input $\mathbf{s}_t$ consists of four adjacent frames of the ATARI game. Each is resized to 84×84 and converted to grayscale. These frames are represented as four channels and processed by an 8×8 convolution with stride four, followed by a 4×4 convolution with stride 2, followed by two fully connected layers. The final output predicts the action value $q[\mathbf{s}_t, a_t]$ for each of the 18 actions in this state.



https://docs.pytorch.org/tutorials/intermediate/reinforcement_q_learning.html

Reinforcement Learning (DQN) Tutorial

Created On: Mar 24, 2017 | Last Updated: Jun 16, 2025 | Last Verified: Nov 05, 2024

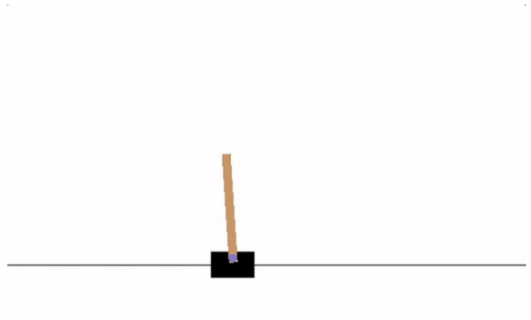
Author: Adam Paszke
Mark Towers

This tutorial shows how to use PyTorch to train a Deep Q Learning (DQN) agent on the CartPole-v1 task from Gymnasium.

You might find it helpful to read the original Deep Q Learning (DQN) paper

Task

The agent has to decide between two actions - moving the cart left or right - so that the pole attached to it stays upright. You can find more information about the environment and other more challenging environments at Gymnasium's website.



For our training update rule, we'll use a fact that every Q function for some policy obeys the Bellman equation:

$$Q^{\pi}(s, a) = r + \gamma Q^{\pi}(s', \pi(s'))$$

The difference between the two sides of the equality is known as the temporal difference error, δ:

$$\delta = Q(s, a) - (r + \gamma \max_{a}' Q(s', a))$$

To minimize this error, we will use the Huber loss. The Huber loss acts like the mean squared error when the error is small, but like the mean absolute error when the error is large - this makes it more robust to outliers when the estimates of Q are very noisy. We calculate this over a batch of transitions, B, sampled from the replay memory:

$$\mathcal{L} = \frac{1}{|B|} \sum_{(s,a,s',r) \in B} \mathcal{L}(\delta)$$

$$\text{where} \quad \mathcal{L}(\delta) = \begin{cases} \frac{1}{2}\delta^2 & \text{for } |\delta| \leq 1, \\ |\delta| - \frac{1}{2} & \text{otherwise.} \end{cases}$$

```python
class DQN(nn.Module):

    def __init__(self, n_observations, n_actions):
        super(DQN, self).__init__()
        self.layer1 = nn.Linear(n_observations, 128)
        self.layer2 = nn.Linear(128, 128)
        self.layer3 = nn.Linear(128, n_actions)

    # Called with either one element to determine next action, or a batch
    # during optimization. Returns tensor([[left0exp,right0exp]...]).
    def forward(self, x):
        x = F.relu(self.layer1(x))
        x = F.relu(self.layer2(x))
        return self.layer3(x)
```

Double Deep-Q

Our networks is used both to predict q_t and select maximum q_{t+1}. This double duty can cause instabilities. One solution is to make two independent networks that each use the other network for max q_{t+1}

$$q_1[s_t, a_t] \leftarrow q_1[s_t, a_t] + \alpha\left(r[s_t, a_t] + \gamma \cdot q_2\left[s_{t+1}, \underset{a}{\operatorname{argmax}}\left[q_1[s_{t+1}, a]\right]\right] - q_1[s_t, a_t]\right)$$

$$q_2[s_t, a_t] \leftarrow q_2[s_t, a_t] + \alpha\left(r[s_t, a_t] + \gamma \cdot q_1\left[s_{t+1}, \underset{a}{\operatorname{argmax}}\left[q_2[s_{t+1}, a]\right]\right] - q_2[s_t, a_t]\right)$$

https://arxiv.org/abs/1912.06680

Dota 2 with Large Scale Deep Reinforcement Learning

OpenAI, *
Christopher Berner, Greg Brockman, Brooke Chan, Vicki Cheung,
Przemysław "Psyho" Dębiak, Christy Dennison, David Farhi, Quirin Fischer,
Shariq Hashme, Chris Hesse, Rafal Józefowicz, Scott Gray, Catherine Olsson,
Jakub Pachocki, Michael Petrov, Henrique Pondé de Oliveira Pinto, Jonathan Raiman,
Tim Salimans, Jeremy Schlatter, Jonas Schneider, Szymon Sidor, Ilya Sutskever, Jie Tang,
Filip Wolski, Susan Zhang

March 10, 2021

Abstract

On April 13th, 2019, OpenAI Five became the first AI system to defeat the world champions at an esports game. The game of Dota 2 presents novel challenges for AI systems such as long time horizons, imperfect information, and complex, continuous state-action spaces, all challenges which will become increasingly central to more capable AI systems. OpenAI Five leveraged existing reinforcement learning techniques, scaled to learn from batches of approximately 2 million frames every 2 seconds. We developed a distributed training system and tools for continual training which allowed us to train OpenAI Five for 10 months. By defeating the Dota 2 world champion (Team OG), OpenAI Five demonstrates that self-play reinforcement learning can achieve superhuman performance on a difficult task.